PRINCIPLES OF LINUX OPERATING SYSTEMS

COURSE LESSONS

PREPARED BY

Abdi Fitah Ahmad Ibraahiim

STUDENT ID

C6240010

CLASS NAME

CNS241

Linux fundamentals, command-line tools and system administration

CHAPTER 1

Introduction to Linux and Ubuntu

Chapter overview

Chapter 1 explains what an operating system is, what Linux means, how Linux developed, how major Linux distribution families differ, and why Ubuntu is a suitable platform for learning Linux fundamentals.

Learning objectives

- Define an operating system and explain its main responsibilities.
- Distinguish the Linux kernel, the GNU user-space tools, and a Linux distribution.
- Describe the historical relationship among Unix, GNU, Linux, Debian, and Ubuntu.
- Compare major Linux distribution families and package managers.
- Explain virtualization, host OS, guest OS, ISO image, and snapshot in a simple way.

1.1 Understanding operating systems

Hardware refers to the physical components of a computer, such as the processor, memory modules, disk, keyboard, display, and network card. Software refers to instructions that tell the computer how to perform useful work. The operating system connects these two sides, ensuring that programs use hardware through managed interfaces rather than competing for direct control.

The difference between the kernel and a user interface is especially important. The kernel performs low-level management; graphical desktop software and command-line shells provide ways for people to control the system. A server can run Linux without a graphical desktop because the kernel and essential services do not require graphical windows.

An operating system (OS) is the main software layer that manages hardware resources and offers services to programs and users. Without an operating system, a computer would not know how to organise its memory, storage, input devices, display, or running programs.

The OS performs several core jobs. It schedules processes so many tasks can share the CPU. It manages memory so one program does not overwrite another. It manages files and directories so data can be stored and found again. It manages devices such as disks, printers, and network cards through drivers. It also provides user interfaces, security controls, and networking support.

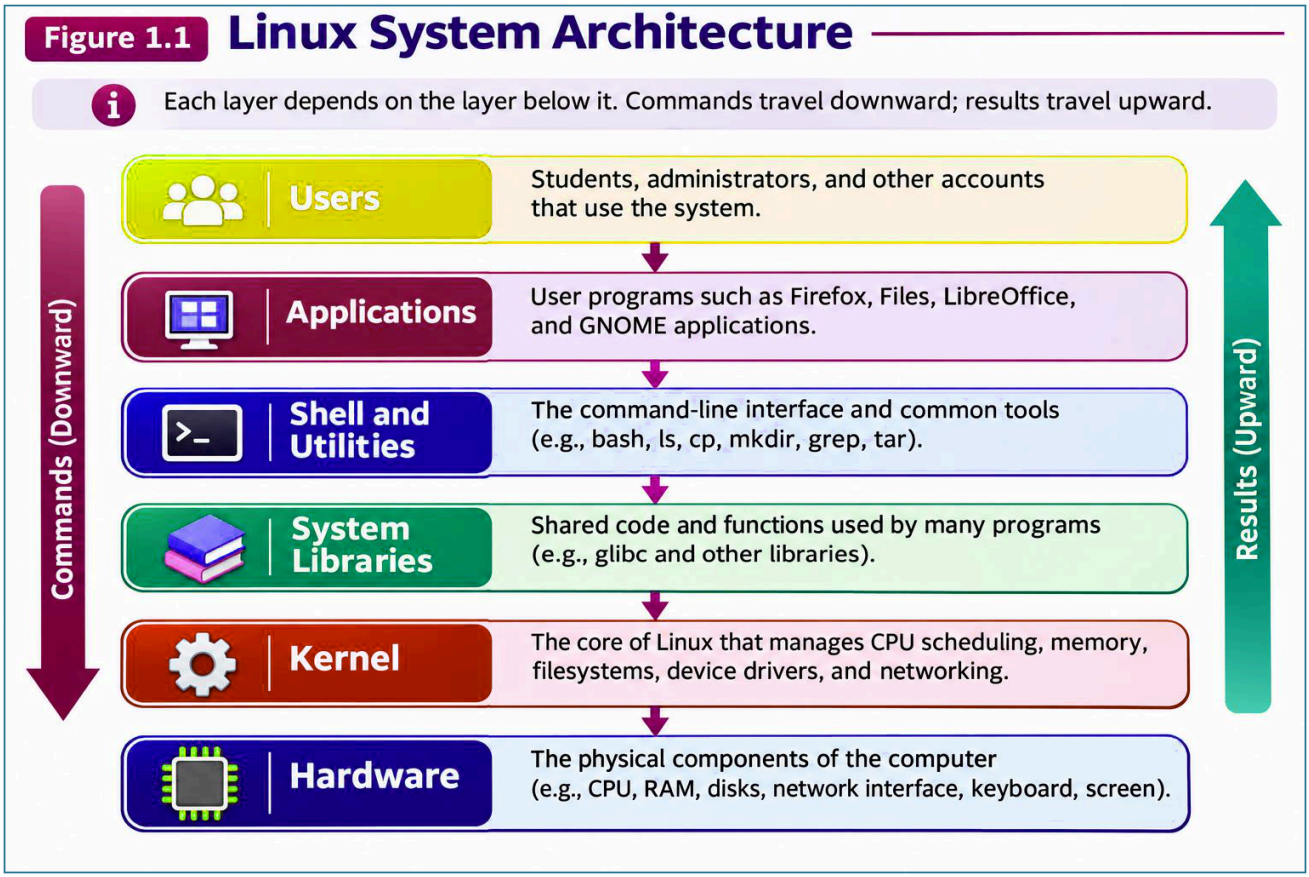


Figure 1.1 Linux system architecture from users down to hardware.

Clear definition: a process is a running program; memory is the working area used while programs are active; a filesystem is the organised structure used to store data on disk.

1.2 Understanding Linux and GNU/Linux

The term GNU/Linux is often used to emphasise that a complete GNU-based Linux system contains the Linux kernel together with GNU tools, compilers, shells, and libraries. Not every Linux-based system uses the full GNU user-space stack; for example, lightweight environments may use alternative tools or C libraries.

Linux supports multiuser operation, which means different people can maintain separate accounts, files, and processes. Multitasking allows multiple programs to make progress concurrently. Access permissions help define who may read, change, or execute information.

Strictly speaking, Linux is the kernel. The kernel is the central core that talks directly to hardware and manages CPU time, memory, devices, filesystems, and networking. In everyday use, people often say “Linux” to mean the full operating system that includes the Linux kernel, the GNU tools, libraries, applications, and a desktop environment.

Linux is important because it is stable, flexible, secure, portable, and widely used. It runs on laptops, desktops, servers, cloud platforms, supercomputers, embedded devices, and network appliances. It is also strongly associated with open-source development, which allows users and organisations to inspect and improve the software.

1.3 History and evolution

Linux developed from a larger historical tradition. Unix contributed the idea of small tools, text files, and a multiuser design. The GNU Project, started by Richard Stallman, aimed to provide a free Unix-like system. Linus Torvalds began the Linux kernel in 1991. Later, distributions such as Debian and Ubuntu combined the kernel with complete software collections that ordinary users could install and update easily.

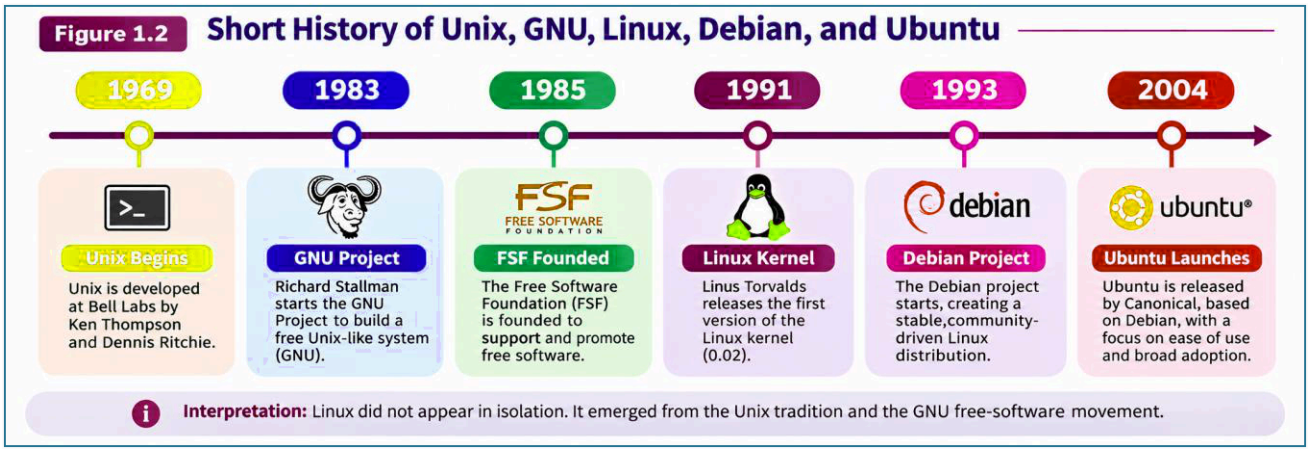


Figure 1.2 A short historical timeline for Unix, GNU, Linux, Debian, and Ubuntu.

1.4 The Linux kernel in simple language

The kernel runs in privileged kernel space. Ordinary applications run in user space. When a program needs a protected service—such as opening a file or creating a process—it makes a system call. The kernel receives that request, checks permissions, and performs the work safely.

Important kernel responsibilities include process scheduling, memory management, filesystem management, device-driver coordination, networking, and security enforcement. A useful way to think about the kernel is this: programs ask, the kernel checks, the hardware responds, and the kernel returns a result.

1.5 Linux distributions and families

Distributions differ mainly in their default software, package management, release schedule, support arrangements, security policies, and administrative conventions. The same Linux kernel concepts apply across these families, while installation and update commands may be different.

Fixed-release distributions generally issue versioned editions and selected updates throughout the support period. Rolling-release distributions deliver ongoing package updates instead of waiting for a full new edition. A rolling release can expose learners to newer software but usually demands more attention to compatibility and maintenance.

A Linux distribution is a complete packaged system built around the Linux kernel. It usually includes system libraries, GNU utilities, a package manager, configured repositories, default applications, and often a desktop environment.

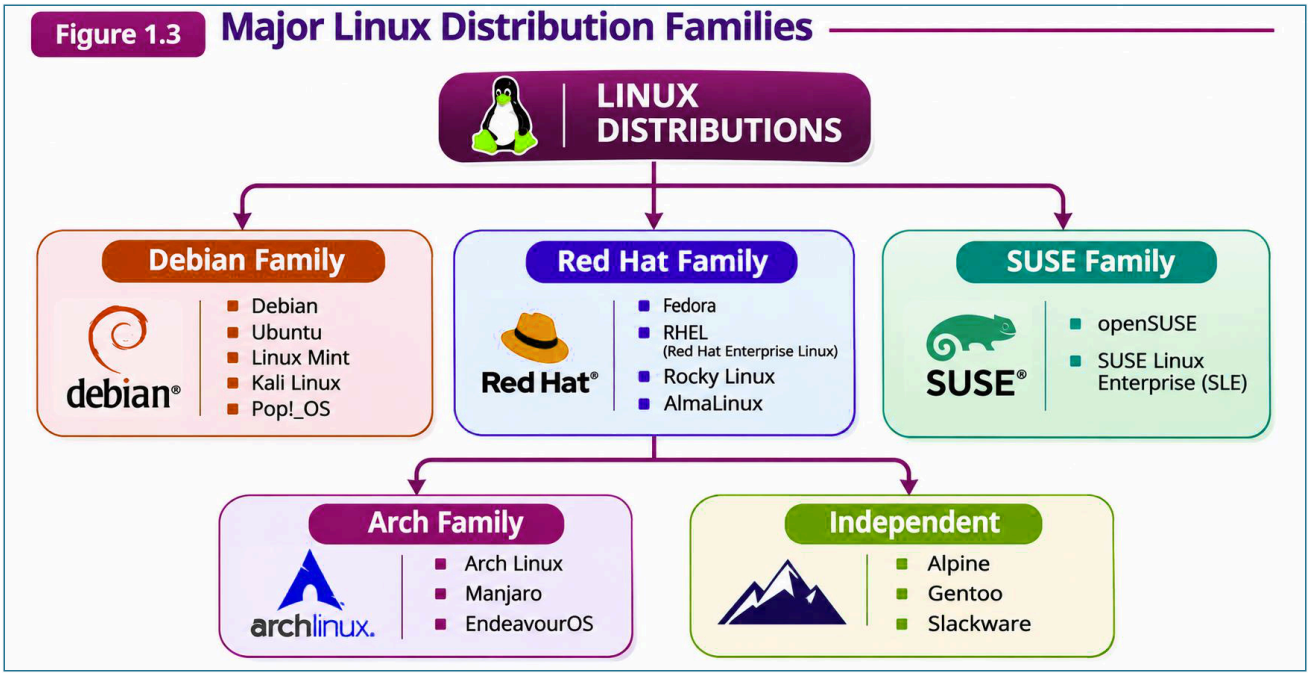


Figure 1.3 Major Linux distribution families and some well-known examples.

Family	Examples	Package manager	General character
Debian-based	Debian, Ubuntu, Mint, Kali	APT / dpkg	Strong documentation, common in teaching
Red Hat family	Fedora, RHEL, Rocky, AlmaLinux	DNF / RPM	Common in enterprise and server administration
SUSE family	openSUSE, SLES	Zypper / RPM	Administrative tools and enterprise use
Arch family	Arch, Manjaro, EndeavourOS	pacman	Rolling release, hands-on control
Independent	Alpine, Gentoo, Slackware	apk, emerge, pkgtools	Specialised or traditional workflows

Comparison in clear language: Ubuntu is easier for beginners than Arch because Ubuntu provides a guided installation and large community support. Fedora often offers newer software earlier than Ubuntu LTS, but Ubuntu LTS emphasises long-term stability. Kali Linux is a specialist security-testing system and is not the best starting point for general Linux learning.

1.6 Ubuntu Linux

Ubuntu is a Debian-based distribution developed with both desktop and server editions. Ubuntu Desktop usually includes a GNOME graphical environment. Ubuntu Server is more commonly managed through the command line. The Long-Term Support (LTS) releases are especially important for teaching because they remain supported for longer and have abundant documentation.

1.7 Linux compared with Windows and macOS

Feature	Linux	Windows	macOS
Licensing	Mostly open source	Proprietary	Proprietary with open-source components

Package management	APT, DNF, pacman, etc.	Mostly graphical installers / winget	App Store, Homebrew, pkg installers
Customisation	Very high	Moderate	Moderate
Common use	Servers, development, cloud, labs	General office, enterprise desktop, gaming	Apple ecosystem, creative work, Unix-like desktop

1.8 Free and open-source software

Free software does not necessarily mean software that costs nothing. It means that users have meaningful freedoms over how the software is run, studied, shared, and changed. Source-code availability and licensing terms determine how those freedoms operate.

The GPL is a copyleft licence. The LGPL has different linking-related conditions, the AGPL adds network-service-related obligations, and permissive licences such as MIT, BSD, and Apache generally allow broad reuse with attribution and other specified conditions. Always read the exact licence and consult qualified advice before making legal decisions.

Free software focuses on users' freedoms to run, study, modify, and share software. Open-source software emphasises access to source code and collaborative development. Many programs belong to both traditions.

Copyleft licences such as the GPL require modified versions to remain under compatible open licences when distributed. Permissive licences such as MIT, BSD, and Apache allow reuse with fewer downstream restrictions. Practical example: a GPL program cannot usually be mixed into a closed-source product in the same easy way that MIT-licensed code can.

1.9 Virtualization and an Ubuntu practice environment

Virtualization allows one physical computer to run another operating system inside a virtual machine (VM). The host OS is the real operating system already installed on the physical machine. The guest OS is the one running inside the VM. The hypervisor is the software that provides virtual hardware to the guest. In this course, VirtualBox is used as the hypervisor.

Review questions

1. Define an operating system and explain four of its responsibilities.
2. Distinguish Linux, GNU/Linux, and a Linux distribution.
3. Describe the historical connection among Unix, GNU, Linux, Debian, and Ubuntu.
4. Compare Ubuntu, Fedora, and Arch Linux in clear practical terms.
5. Explain the difference between a host OS, a guest OS, and a hypervisor.

CHAPTER 2

Linux System Architecture, Command Line, and File Management

Chapter overview

This chapter explains how the Linux command line works, where important directories are found, how commands are built, and how files and folders are created, copied, moved, inspected, and removed safely.

Learning objectives

- Identify important directories in the Linux filesystem hierarchy.
- Break a command into the executable name, its options, and its arguments.
- Use Bash help, history, quoting, and environment variables correctly.
- Manage files and directories with safe command-line practices.

2.1 The Linux filesystem hierarchy

Linux stores everything under one directory tree that begins at /, called the root directory. Unlike Windows, Linux does not normally use drive letters such as C: or D:. Instead, disks and other storage devices appear somewhere within the single tree.

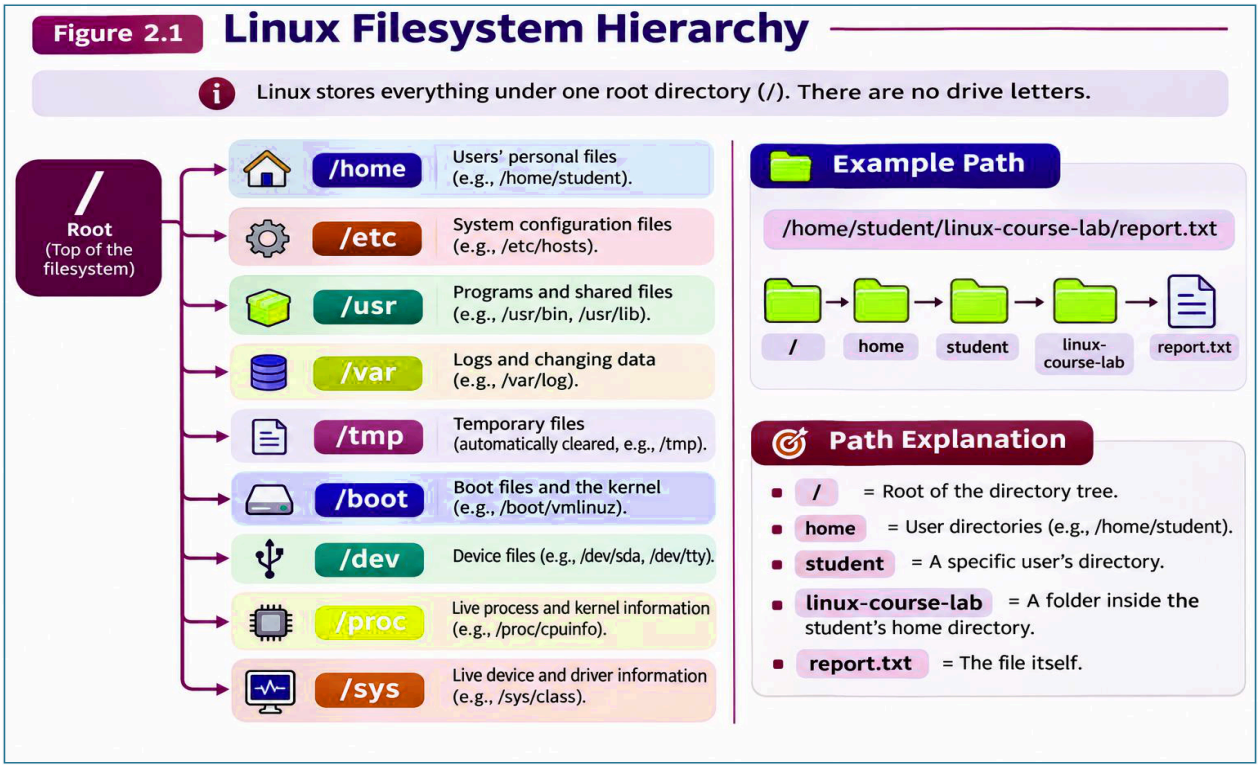


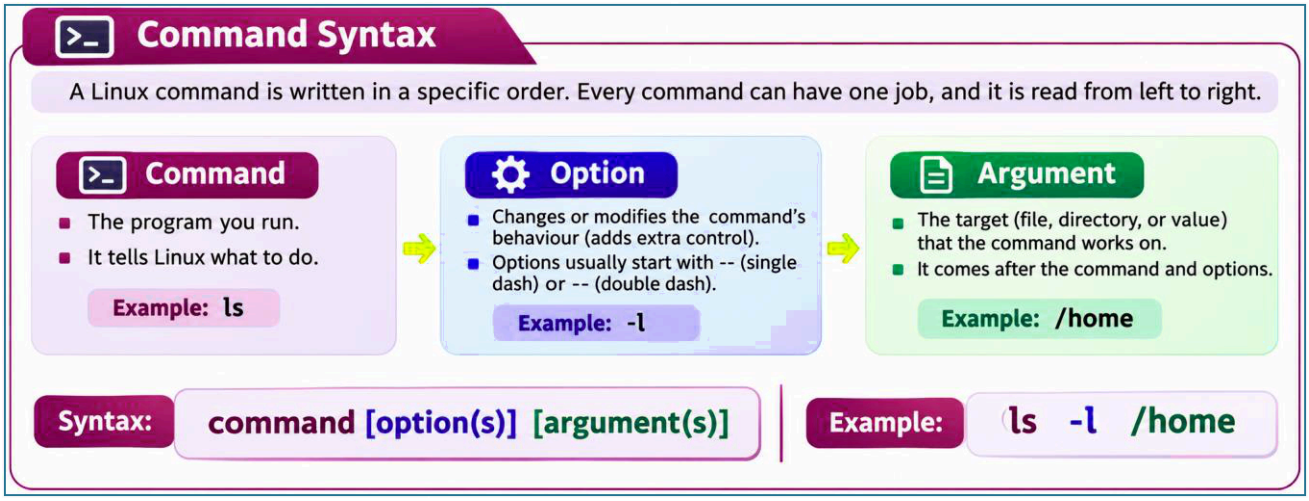
Figure 2.1 Important directories in the Linux filesystem hierarchy.

Directory	Meaning and typical contents
/	The root of the entire filesystem tree
/home	Personal files for ordinary users, e.g., /home/student
/root	Home directory for the root administrator account
/etc	System-wide configuration files
/usr	Installed programs and read-mostly shared files
/var	Changing data such as logs, queues, caches
/tmp	Temporary files
/boot	Kernel and bootloader files
/dev	Device files
/proc	Live kernel and process information built in memory
/sys	Live device and driver information built in memory

2.2 Terminal, shell, and Bash

The terminal emulator is the window on your screen. The shell is the program inside that window that reads your commands. Bash is the shell used in this course. When you type a command and press Enter, Bash separates the command into parts, finds the executable program, and runs it.

2.3 Command syntax and special symbols



Consider the command **ls -lah** ~/Documents. The word ls is the command name; **-l** requests long listing, **-a** includes hidden files, and **-h** makes sizes easier to read. The final expression ~/Documents is the directory argument. Options tell a command how to work, while arguments identify the data it should work on.

An option that takes a value must receive the right type of argument. For instance, **head -n 5** notes.txt uses 5 as the number of lines and notes.txt as the input filename. Likewise, **grep -i** error app.log uses **-i** to ignore letter case, error as the search pattern, and app.log as the file to search.

Most Linux commands follow this pattern: **command [options] [arguments]**. The command is the program name. Options change how the program behaves. Arguments tell the program what object to act on, such as a file or directory name.

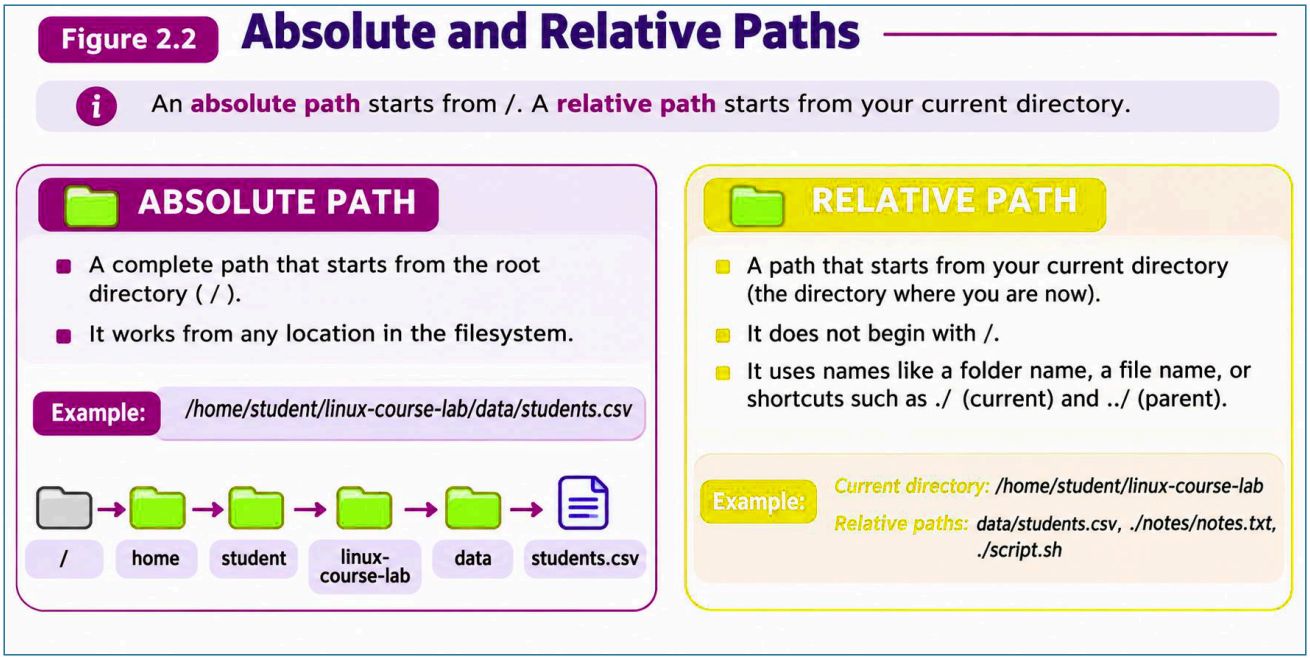


Figure 2.2 Absolute and relative paths.

- / means the root of the filesystem.
- . means the current directory.
- .. means the parent directory.
- ~ means the current user's home directory.
- * matches many filenames; ? matches one character.
- $ introduces variable expansion, such as $HOME or $PATH.
- | sends one command's output into another command.

2.4 Getting help

2.4.1 man

Definition

The man command opens the manual pages installed on a Linux computer. A manual page explains what a command does, the correct order of its arguments, the options it accepts, and sometimes practical examples. Use man when a new command or unfamiliar flag appears in your course. Inside the viewer, press Space to move forward, type / followed by a word to search, and press q to quit.

Why students use this command

Read a manual page when you need an accurate explanation of an unfamiliar option rather than guessing.

Syntax

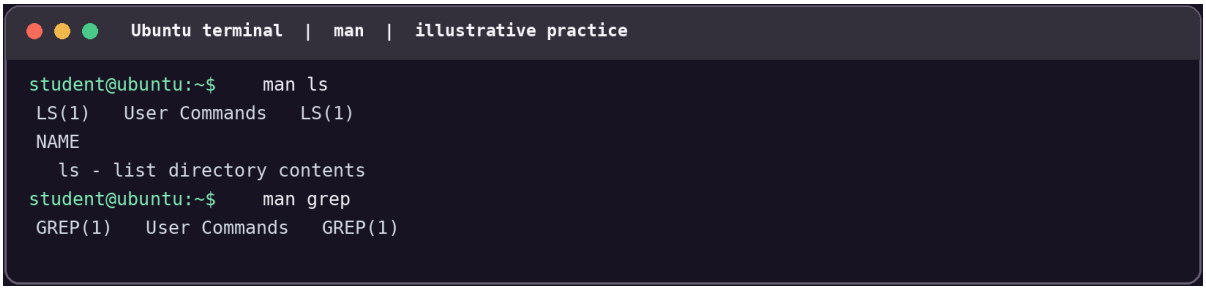
```
man [section] command
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage

-k keyword	Search manual descriptions	Function: search manual descriptions. Example: man -k archive
-f command	Show a short manual-page description	Function: show a short manual-page description. Example: man -f ls

Practice screenshot — test commands and example output



Important note. Man pages are viewed in a pager. The page is not a command to run; it is documentation.

2.4.2 --help

Definition

The --help option asks a command to display a short usage guide in the terminal. It is not a command by itself: you attach it to a program, as in ls --help or grep --help. The output usually lists accepted options and how to use them. Some shell built-ins have different help mechanisms.

Why students use this command

Use it for a quick reminder of syntax and common flags.

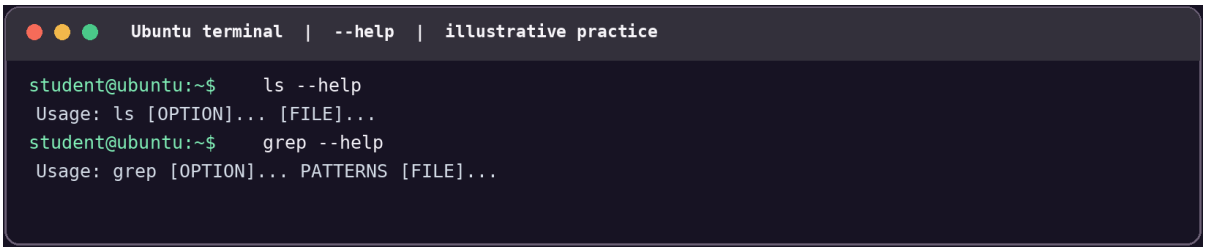
Syntax

```
command --help
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
--help	Show built-in usage text	Function: show built-in usage text. Example: ls --help

Practice screenshot — test commands and example output



Important note. Some programs and Bash built-ins use help differently; use man or help when --help is unavailable.

2.4.3 help

Definition

The help command is part of Bash. It describes commands built into the shell, such as cd, echo, and export. It is useful because these shell built-ins may not be separate executable programs. Type help followed by the built-in name to read its syntax and available flags.

Why students use this command

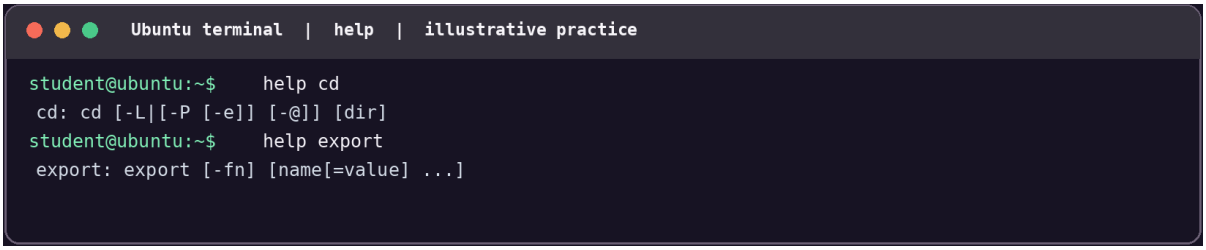
Use it when the command is part of Bash instead of a separate executable.

Syntax

```
help [builtin]
```

Options: The examples below use the command without additional flags.

Practice screenshot — test commands and example output



2.4.4 type

Definition

The type command tells you how Bash understands a command name. It may identify that name as a shell built-in, alias, function, or executable program on disk. This helps explain why two similarly named commands sometimes behave differently and which one the terminal will run.

Why students use this command

Use it when you need to know what Bash will actually run.

Syntax

```
type name
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-a	**Show all known ways to resolve the name**	**Function: show all known ways to resolve the name.** **Example: type -a ls**

Practice screenshot — test commands and example output



2.4.5 which

Definition

The which command looks through the directories in PATH to locate an executable program. It normally prints the filesystem path of the matching program, such as /usr/bin/ls. It does not fully describe shell aliases or built-ins, so type or command -v can be more informative.

Why students use this command

Use it to locate installed external programs.

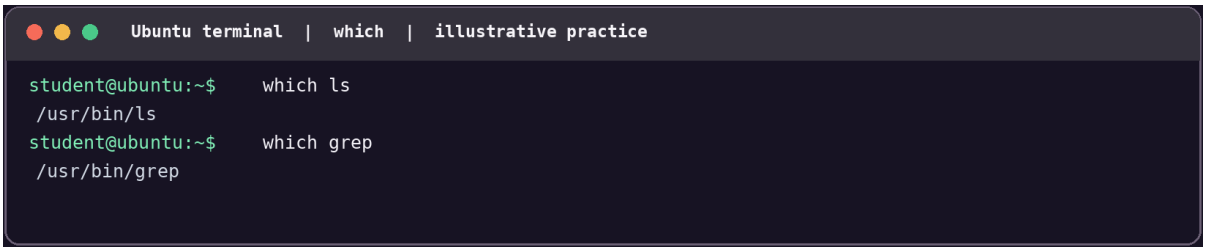
Syntax

```
which program
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-a`	Show all matching executables in PATH	Function: show all matching executables in PATH. Example: which -a python3

Practice screenshot — test commands and example output



2.4.6 whereis

Definition

The whereis command searches common system locations for a program's executable file, source files, and manual pages. It is useful when you know a program's name but want to find related installed files. The result lists paths, not the contents of those files.

Why students use this command

Use it to locate related files for known system tools.

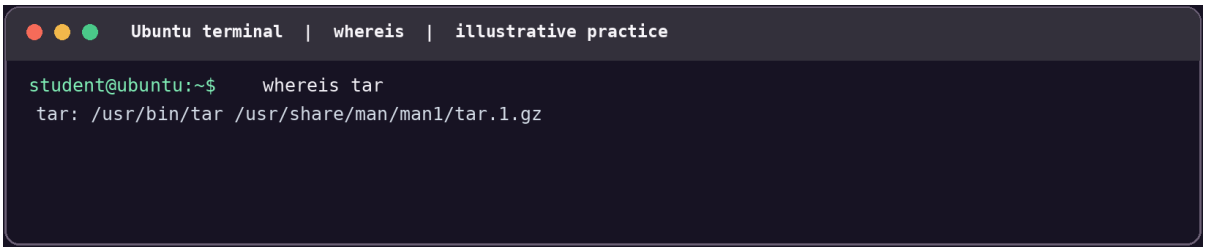
Syntax

```
whereis command
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-b`	Look for binaries only	Function: look for binaries only. Example: whereis -b bash
`-m`	Look for manual pages	Function: look for manual pages. Example: whereis -m ls

Practice screenshot — test commands and example output



2.4.7 command -v

Definition

The command -v expression asks Bash how a name would be resolved when used as a command. It can identify executables, shell built-ins, functions, and aliases. Students commonly use it to check whether a tool is available before trying to run it.

Why students use this command

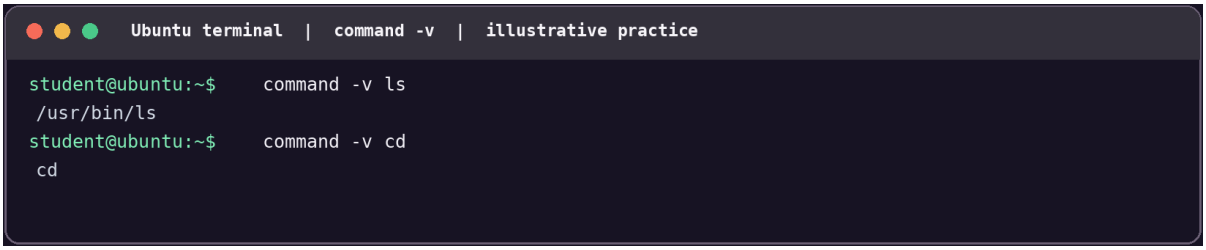
Use it to check command availability before writing a script.

Syntax

```
command -v name
```

Options: The examples below use the command without additional flags.

Practice screenshot — test commands and example output



2.5 History, quoting, and environment variables

Command history remembers what you typed earlier. Press the Up Arrow to move backward through previous commands. Tab completion saves typing and reduces mistakes. Quoting matters because the shell interprets spaces and special characters. Single quotes preserve text almost exactly. Double quotes still allow variable expansion. A backslash (\) escapes the next special character.

Environment variables store values used by the shell and programs. Common examples include HOME, USER, SHELL, PWD, and PATH. PATH tells the shell where to search for executable commands.

2.5.1 echo

Definition

The echo command writes text and expanded shell-variable values to standard output, which normally means the terminal screen. Students use it to display simple messages and to inspect values such as $HOME. Quotation marks help control how spaces and special characters are processed before echo receives the text.

Why students use this command

Use it for quick messages and inspecting simple variable values. For reliably formatted file creation, prefer printf.

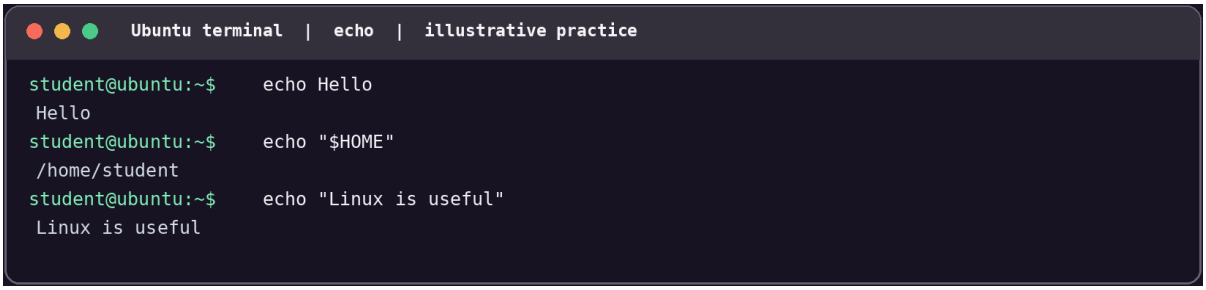
Syntax

`echo [text ...]`

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-n	Do not append a newline (shell-dependent details)	Function: do not append a newline (shell-dependent details). Example: echo -n Hello

Practice screenshot — test commands and example output



2.5.2 printenv

Definition

The printenv command displays environment variables and their values. These variables carry settings to programs started from a shell, such as HOME, PATH, and SHELL. Give printenv a variable name to show one value, or run it without a name to list exported environment values.

Why students use this command

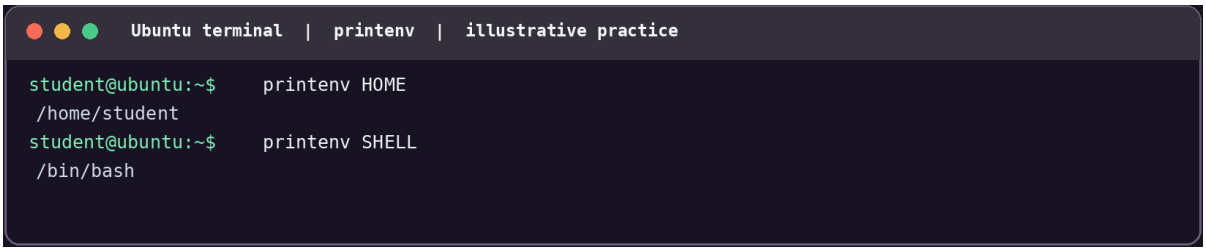
Use it to check PATH, HOME or other environment settings.

Syntax

`printenv [VARIABLE]`

Options: The examples below use the command without additional flags.

Practice screenshot — test commands and example output



2.5.3 export

Definition

The export command marks a shell variable so that programs launched from the current shell can read its value. For example, export COURSE=Linux makes COURSE available to commands started afterward. The value normally lasts only for the current session unless it is added to a shell startup configuration.

Why students use this command

Use it when a tool needs a configuration value through its environment.

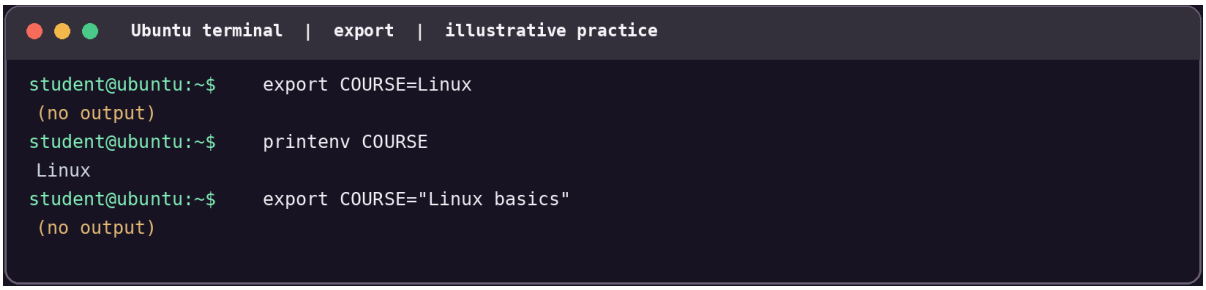
Syntax

```
export NAME=value
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-p	Show exported variables in shell notation	Function: show exported variables in shell notation. Example: export -p

Practice screenshot — test commands and example output



2.6 File and directory management commands

2.6.1 pwd

Definition

The pwd (print working directory) command shows the full path of the directory in which the shell is currently working. Knowing that location matters because relative file paths are interpreted from it. Run pwd before copying, moving, or deleting files when you are unsure where you are.

Why students use this command

Relative paths are interpreted from that location. Check pwd before moving or deleting files.

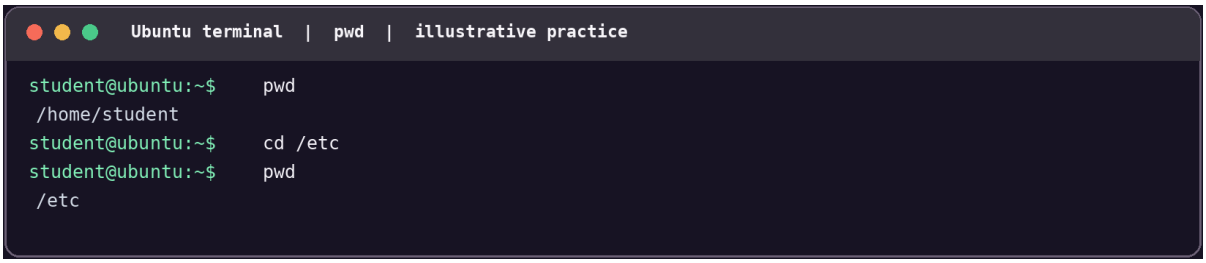
Syntax

```
pwd [options]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-P`	Display the physical directory path with symbolic links resolved	Function: display the physical directory path with symbolic links resolved. Example: pwd -P
`-L`	Display the logical path; default in Bash	Function: display the logical path; default in Bash. Example: pwd -L

Practice screenshot — test commands and example output



2.6.2 ls

Definition

The ls (list) command is used in Linux and Unix-like systems to list files and directories in the current working directory or at a specified path. Without options, it usually prints their names. With options such as -l, -a, and -h, it can show permissions, ownership, file size, dates, and hidden entries. Students use ls before working with a folder and afterward to confirm that a file was created, copied, renamed, or removed.

Why students use this command

Use it to see available files and confirm the result of other commands.

Syntax

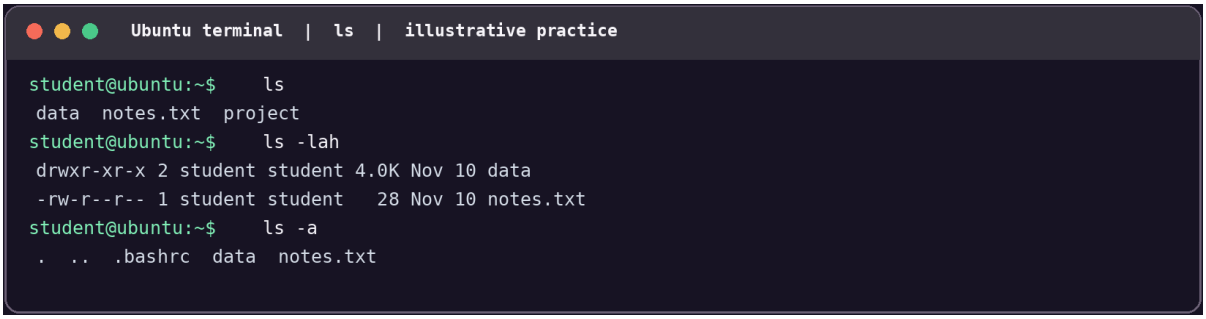
```
ls [options] [path]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-l`	Long Format	Displays detailed file information, including permissions, number of links, owner, group, file size, and modification time. Example: ls -l
`-a`	All Files	Lists all entries, including hidden files and directories whose names begin with a dot (.). Example: ls -a
`-h`	Human-Readable	Works with -l to show sizes using readable units such as K, M, and G. Example: ls -lh

`-R`	Recursive Listing	Lists a directory and then visits its subdirectories, showing their contents too. Example: ls -R
`-t`	Sort by Time	Sorts entries according to modification time, showing the newest first. Example: ls -lt
`-S`	Sort by Size	Sorts entries by file size, usually showing the largest first. Example: ls -lS

Practice screenshot — test commands and example output



2.6.3 cd

Definition

The cd (change directory) command changes the current working directory of your Bash session. For example, cd Documents moves into the Documents folder, while cd .. moves to its parent folder. This command does not copy or delete data: it changes where the shell looks when you use relative paths.

Why students use this command

Use it to move around the filesystem before working with files.

Syntax

```
cd [directory]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`..`	Parent directory	Function: parent directory. Example: cd ..
`~`	Home directory	Function: home directory. Example: cd ~
`-`	Previous directory	Function: previous directory. Example: cd -

Practice screenshot — test commands and example output



2.6.4 mkdir

Definition

The mkdir (make directory) command creates a new directory so you can organise files into folders. You can create one folder or multiple folders with a single command. With -p, mkdir also creates any missing parent directories along a path, making it useful for building a project structure.

Why students use this command

Use it to organise work into folders such as data, reports and backups.

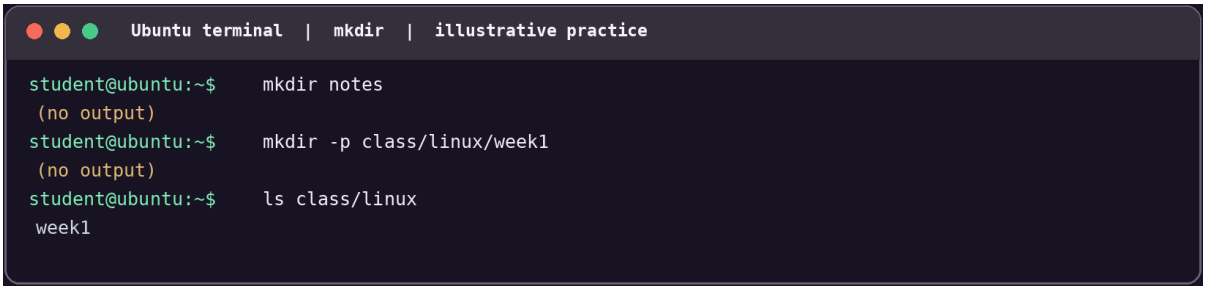
Syntax

```
mkdir [options] directory...
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-p`	Create missing parent directories; do not fail if a directory already exists	Function: create missing parent directories; do not fail if a directory already exists. Example: mkdir -p course/week1
`-v`	Describe the directories created	Function: describe the directories created. Example: mkdir -v projects

Practice screenshot — test commands and example output



2.6.5 rmdir

Definition

The rmdir (remove directory) command deletes an empty directory. It checks that the directory contains no files or subfolders; if it is not empty, the command refuses to remove it. This makes rmdir a safer choice for cleaning up unused empty folders than a recursive deletion command.

Why students use this command

Use it for safe cleanup of unused empty folders.

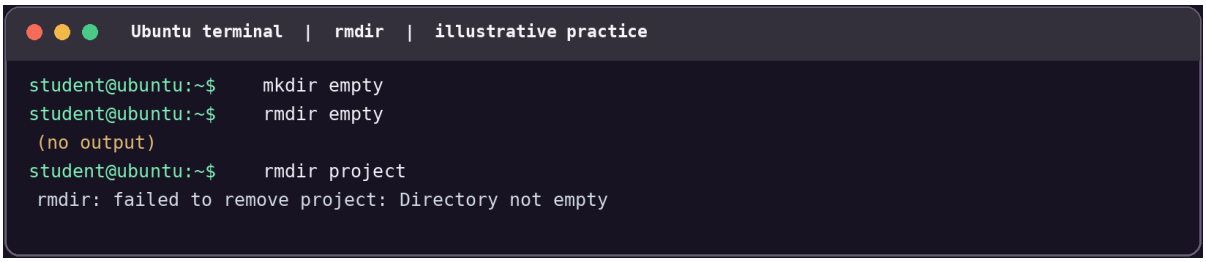
Syntax

```
rmdir [options] directory...
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-p`	Remove directory and empty parents, as far as possible	Function: remove directory and empty parents, as far as possible. Example: rmdir -p old/subdir

Practice screenshot — test commands and example output



2.6.6 touch

Definition

The touch command updates the access and modification timestamps of files. If a named file does not exist, touch normally creates an empty file first. It is useful when preparing simple practice files, but it does not open an editor or write text into those files.

Why students use this command

Use it to create harmless sample files before learning file management.

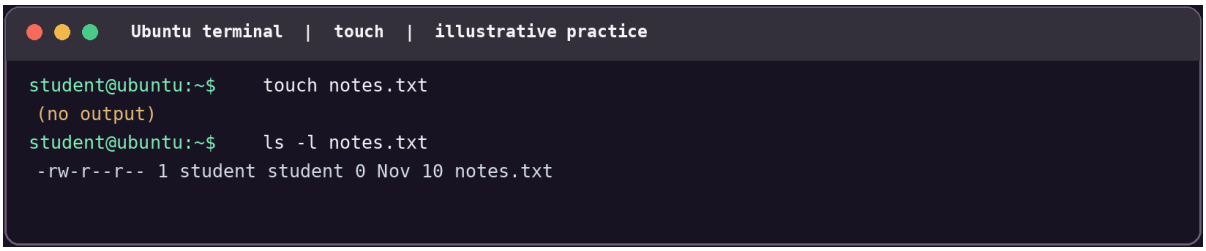
Syntax

```
touch [options] file...
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-c`	Do not create files that do not already exist	Function: do not create files that do not already exist. Example: touch -c absent.txt
`-t TIME`	Set a specific timestamp using the required numeric format	Function: set a specific timestamp using the required numeric format. Example: touch -t 202601011200 notes.txt

Practice screenshot — test commands and example output



2.6.7 cp

Definition

The cp (copy) command creates a duplicate of a file or directory at another location or under a different name. The original remains available after a successful copy. Use -r when you need to copy an entire directory and -i when you want confirmation before an existing destination is replaced.

Why students use this command

Use it to create backup copies or duplicate project files.

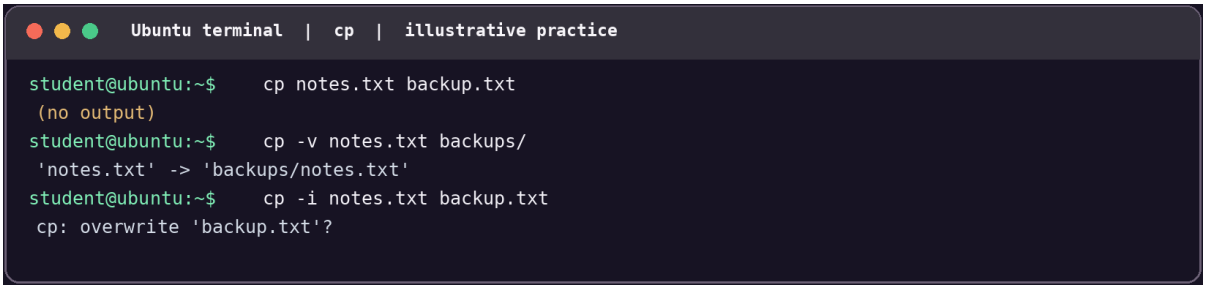
Syntax

```
cp [options] SOURCE DESTINATION
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-r`	Copy a directory and its contents recursively	Function: copy a directory and its contents recursively. Example: cp -r data/ data-copy/
`-i`	Ask before replacing an existing target	Function: ask before replacing an existing target. Example: cp -i a.txt b.txt
`-v`	Print source and destination names while copying	Function: print source and destination names while copying. Example: cp -v a.txt backups/

Practice screenshot — test commands and example output



2.6.8 mv

Definition

The mv (move) command changes the name or location of a file or directory. For example, it can rename draft.txt to final.txt or move final.txt into a folder called reports. Unlike cp, mv does not normally leave another copy at the source location. It may overwrite an existing destination, so -i is useful for safety.

Why students use this command

Use it to organise or rename work without duplication.

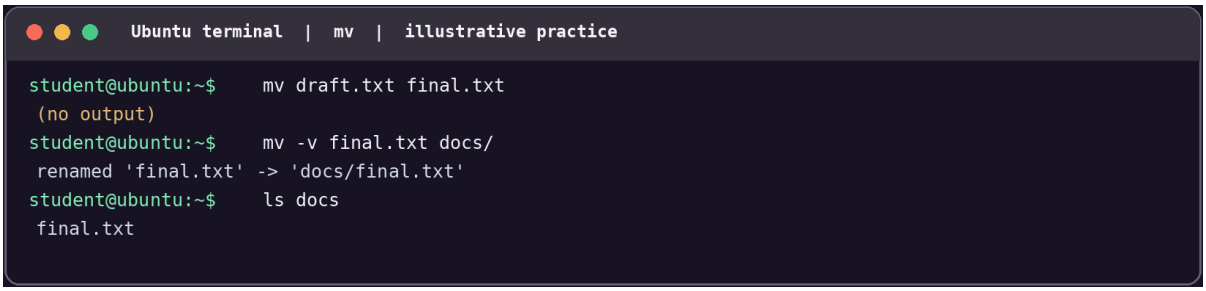
Syntax

```
mv [options] SOURCE DESTINATION
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-i`	Ask before overwriting a destination	Function: ask before overwriting a destination. Example: mv -i old.txt final.txt
`-v`	Print each completed move	Function: print each completed move. Example: mv -v old.txt docs/

Practice screenshot — test commands and example output



2.6.9 rm

Definition

The rm (remove) command deletes named files and, when requested, directories. Files removed with rm do not normally enter the desktop Trash folder. The -i option asks before deletion, while -r descends into directory contents. Always verify the target before using rm; a wrong path can destroy important work.

Why students use this command

Use it only after confirming the target because accidental removal can be permanent.

Syntax

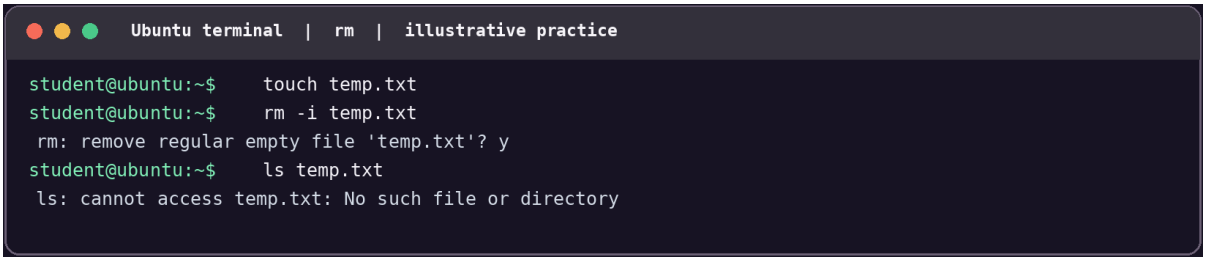
```
rm [options] target...
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-i`	Ask before each removal	Function: ask before each removal. Example: rm -i temp.txt

`-r`	Remove a directory recursively, including its files	Function: remove a directory recursively, including its files. Example: rm -r scratch/
`-f`	Force operation and suppress some prompts/errors; use with extreme care	Function: force operation and suppress some prompts/errors; use with extreme care. Example: rm -f temp.txt

Practice screenshot — test commands and example output



Important note. Do not use rm -rf on unknown directories. Always run pwd and ls first.

CHAPTER 3 — Text Processing, Searching, Archiving, and Editors

Chapter overview

Chapter 3 teaches you how to inspect text files, search for names or patterns, combine commands with pipes, process lines of text, create compressed archives, and edit files with Nano and Vim.

Learning objectives

- View short and long text files efficiently.
- Search for files by name and for content by text pattern.
- Use redirection and pipes to combine commands.
- Create, inspect, compress, and extract archives.
- Edit files with Nano and understand the basic modes of Vim.
- Distinguish hard links from symbolic links.

3.1 Viewing files and file information

3.1.1 cat

Definition

The cat (concatenate) command reads files and writes their contents to standard output. This makes it useful for quickly viewing small text files and combining multiple files into a continuous stream. For long documents, less is usually easier to read. Options such as -n can add line numbers.

Why students use this command

Use it for reading short files; long files are easier to inspect with less.

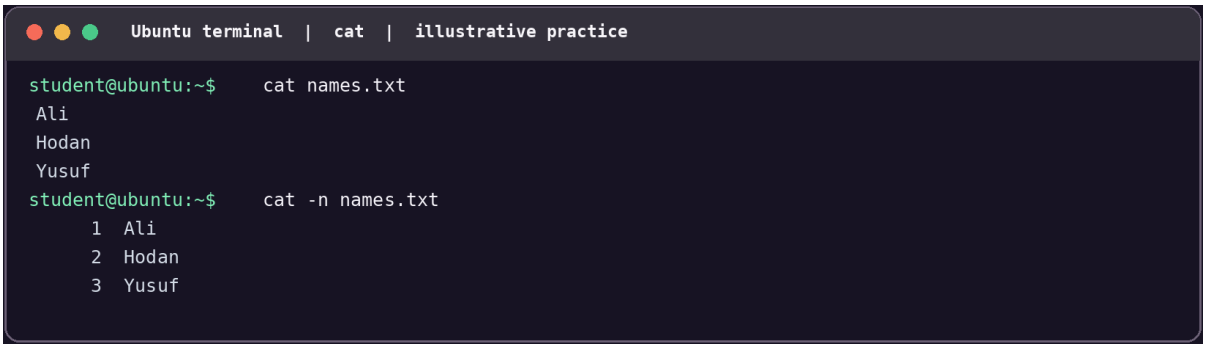
Syntax

```
cat [options] file...
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-n`	Add numbers to all output lines	Function: add numbers to all output lines. Example: cat -n names.txt
`-b`	Number non-empty lines	Function: number non-empty lines. Example: cat -b names.txt

Practice screenshot — test commands and example output



3.1.2 less

Definition

The less command opens a text file or command output one screen at a time. It is called a pager because you can move through the material without printing everything at once. Use Space to advance, b to go backward, / to search for text, and q to quit. It does not modify the file.

Why students use this command

Use it to read logs, manuals and long reports comfortably.

Syntax

```
less [options] file
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-N	Display line numbers	Function: display line numbers. Example: less -N notes.txt
-S	Do not wrap long lines; allow horizontal viewing	Function: do not wrap long lines; allow horizontal viewing. Example: less -S app.log

Practice screenshot — test commands and example output



3.1.3 more

Definition

The more command displays a long file one screen at a time. It offers basic paging and is common on Unix-like systems. Press Space to move forward and q to leave the viewer. It is less flexible than less but useful when less is unavailable.

Why students use this command

Use it if less is unavailable or for basic sequential reading.

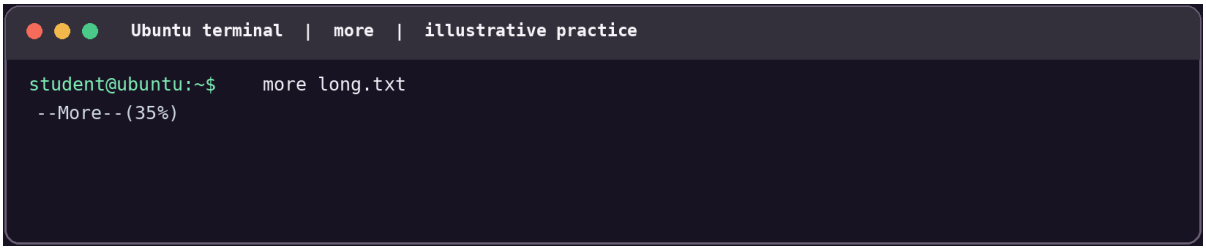
Syntax

```
more file
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-d	Display helpful prompts at the bottom on supported versions	Function: display helpful prompts at the bottom on supported versions. Example: more -d long.txt

Practice screenshot — test commands and example output



3.1.4 head

Definition

The head command displays the beginning of a file or input stream. By default, it prints the first ten lines. Use -n to choose a specific number of lines, such as the first five rows of a dataset. It is useful for checking a file's header without opening the whole file.

Why students use this command

Use it to preview file headers and inspect the first records of a dataset.

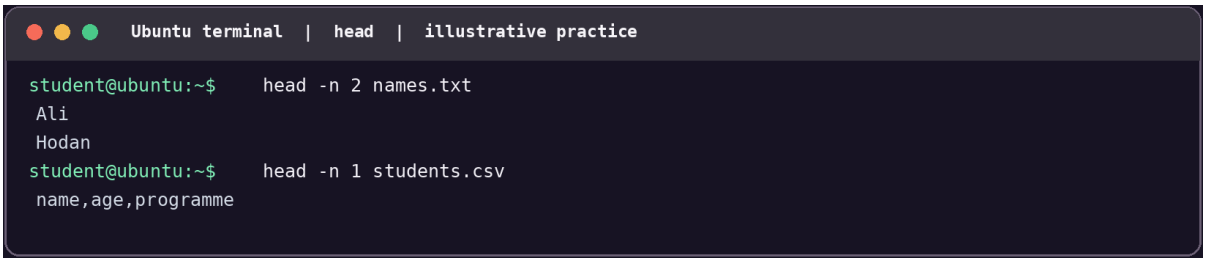
Syntax

```
head [options] file
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-n N	Show the first N lines	Function: show the first N lines. Example: head -n 3 names.txt
-c N	Show the first N bytes	Function: show the first N bytes. Example: head -c 10 notes.txt

Practice screenshot — test commands and example output



3.1.5 tail

Definition

The tail command displays the end of a file or input stream. By default, it prints the last ten lines. It is especially useful for reading the newest log messages, because recent entries usually appear at the bottom. With -f, tail continues showing new lines until you press Ctrl+C.

Why students use this command

Use it for recent log entries and monitoring activity.

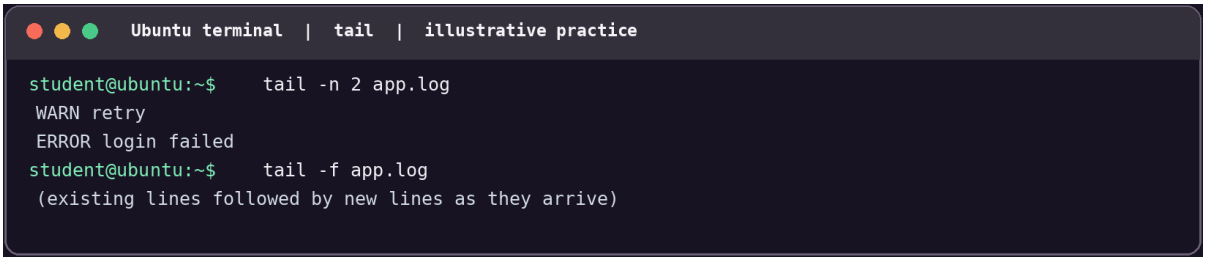
Syntax

`tail [options] file`

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-n N`	Show the last N lines	Function: show the last N lines. Example: tail -n 2 app.log
`-f`	Keep following appended lines	Function: keep following appended lines. Example: tail -f app.log
`-F`	Follow by file name across file replacement or rotation	Function: follow by file name across file replacement or rotation. Example: tail -F app.log

Practice screenshot — test commands and example output



3.1.6 file

Definition

The file command examines data to estimate what kind of file it is, such as plain text, an image, a PDF, or a compressed archive. It does not rely only on the filename extension. Use it when a downloaded file has an unfamiliar or missing extension.

Why students use this command

Use it when checking unknown downloads or files without a useful extension.

Syntax

```
file [options] path
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-b`	Show only the description, without the filename	Function: show only the description, without the filename. Example: file -b notes.txt
`-i`	Show MIME-style type information	Function: show MIME-style type information. Example: file -i notes.txt

Practice screenshot — test commands and example output



3.1.7 stat

Definition

The stat command displays filesystem metadata about a file or directory. Its output can include byte size, permissions, owner, inode number, and access or modification times. Use stat when you need more exact information than a basic ls listing provides.

Why students use this command

Use it when you need more detail than ls -l provides.

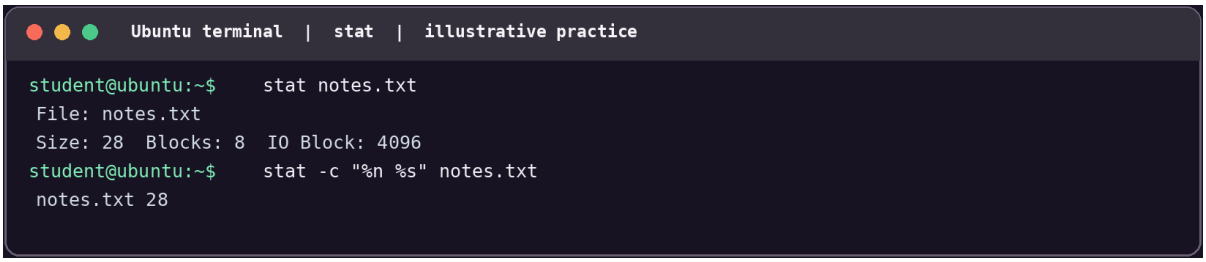
Syntax

```
stat [options] path
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-c FORMAT`	Select displayed metadata fields	Function: select displayed metadata fields. Example: stat -c "%n %s" notes.txt

Practice screenshot — test commands and example output



3.2 Searching files and text

3.2.1 find

Definition

The find command searches directory trees in real time for files and folders that match conditions you supply. You choose a starting directory, then tests such as -name for filenames, -type for file kinds, or -size for sizes. Unlike an indexed search, find examines the filesystem as it exists now.

Why students use this command

Use it for accurate searching including newly created files.

Syntax

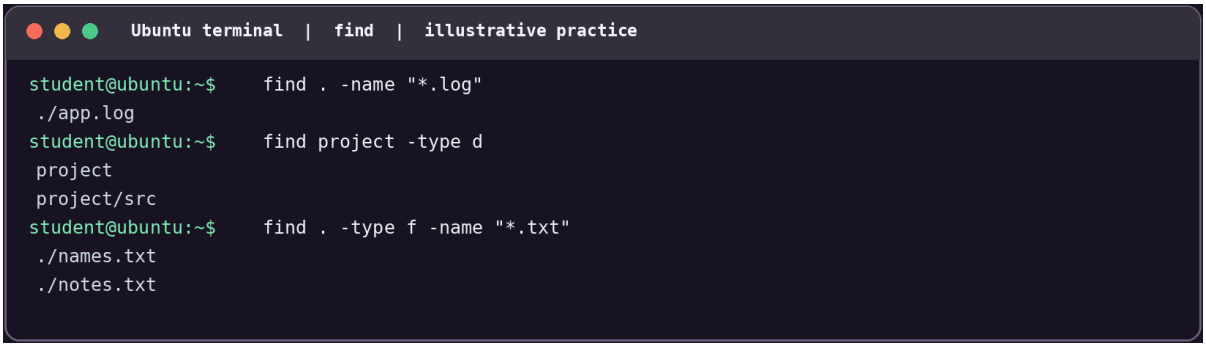
```
find STARTPATH [tests] [actions]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-name PATTERN`	Case-sensitive filename pattern	Function: case-sensitive filename pattern. Example: find . -name "*.txt"
`-iname PATTERN`	Case-insensitive filename pattern	Function: case-insensitive filename pattern. Example: find . -iname "*.TXT"
`-type f`	Only regular files	Function: only regular files. Example: find . -type f
`-type d`	Only directories	Function: only directories. Example: find . -type d

Option / Shortcut	Name	Description & Example Usage
`-size +1M`	Files larger than 1 MiB under find's size semantics	Function: files larger than 1 MiB under find's size semantics. Example: find . -size +1M
`-mmin -30`	Modified in approximately the past 30 minutes	Function: modified in approximately the past 30 minutes. Example: find . -mmin -30

Practice screenshot — test commands and example output



Important note. Put filename patterns in quotes so Bash does not expand them before find receives them.

3.2.2 locate

Definition

The locate command searches a previously built database of file paths. Because it does not need to walk the entire directory tree, it is usually fast. However, recently created files may be missing until the database is updated. It searches filenames and paths, not text inside files.

Why students use this command

Use it for a fast broad path search when a slightly outdated result is acceptable.

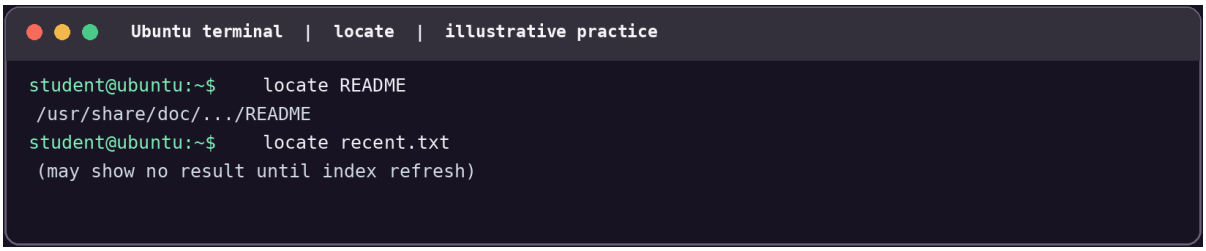
Syntax

```
locate PATTERN
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-i`	Ignore letter case	Function: ignore letter case. Example: locate -i README
`-n N`	Limit the number of results	Function: limit the number of results. Example: locate -n 5 README

Practice screenshot — test commands and example output



3.2.3 updatedb

Definition

The updatedb command refreshes the filename index used by locate or plocate. Updating the index helps recent file changes appear in later locate searches. It may require administrator permission and may not be installed in a minimal Ubuntu environment.

Why students use this command

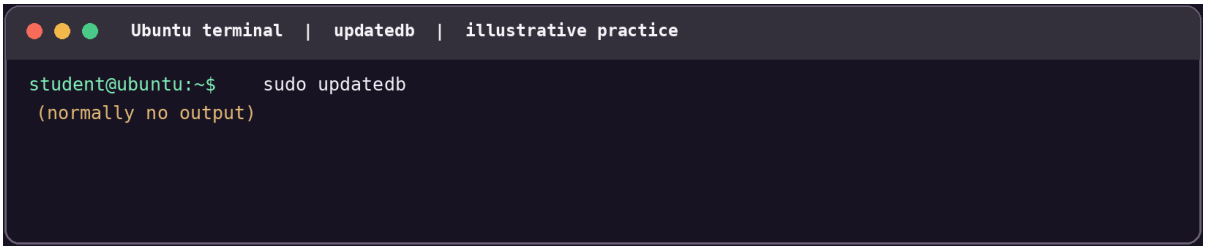
Use it when locate misses recent file additions and you are authorised to refresh the index.

Syntax

```
sudo updatedb
```

Options: The examples below use the command without additional flags.

Practice screenshot — test commands and example output



Important note. This command may be unavailable on minimal Ubuntu installations.

3.2.4 grep

Definition

The grep (global regular expression print) command searches text for a word or pattern and prints the lines that match. It can search a single file, multiple files, or text received through a pipe. By default matching is case-sensitive; use -i to ignore letter case and -n to show matching line numbers.

Why students use this command

Use it to locate words in documents, filters, source code and logs.

Syntax

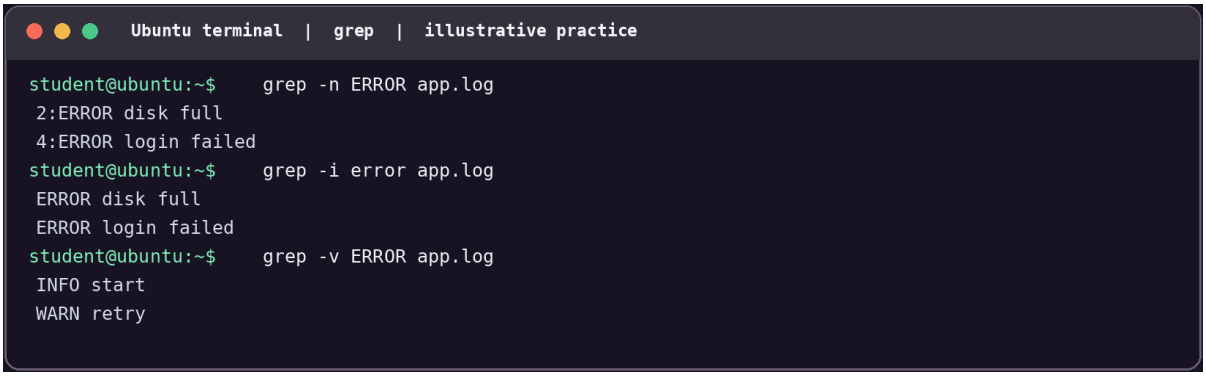
```
grep [options] PATTERN FILE
```

Options and shortcuts

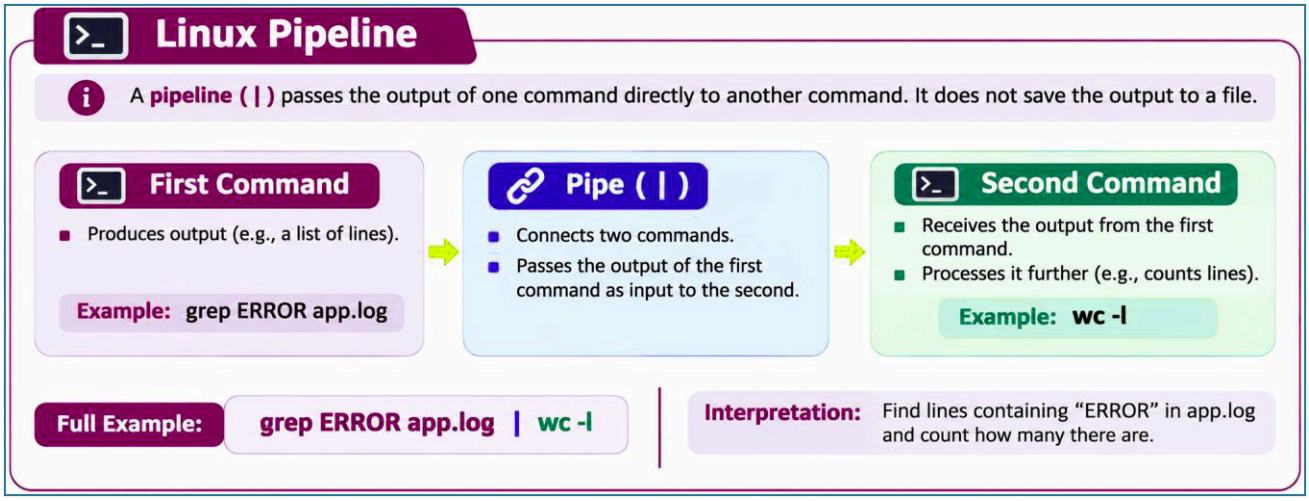
Option / Shortcut	Name	Description & Example Usage

`-i`	Ignore uppercase/lowercase differences	Function: ignore uppercase/lowercase differences. Example: grep -i error app.log
`-n`	Print matching line numbers	Function: print matching line numbers. Example: grep -n ERROR app.log
`-r`	Search files in subdirectories	Function: search files in subdirectories. Example: grep -r TODO project/
`-v`	Print lines that do not match	Function: print lines that do not match. Example: grep -v ERROR app.log
`-c`	Count matching lines	Function: count matching lines. Example: grep -c ERROR app.log
`-l`	Print matching filenames only	Function: print matching filenames only. Example: grep -l TODO *.txt

Practice screenshot — test commands and example output



3.3 Redirection and pipes



Every normal Linux process begins with three standard streams: standard input (file descriptor 0), standard output (file descriptor 1), and standard error (file descriptor 2). Standard output carries normal

results, while standard error carries diagnostic messages. These streams are separate even if a terminal displays both.

The > operator replaces a file with the command's output; >> appends to it. The < operator supplies input from a file. The 2> operator writes error output into a file. A pipe, written |, connects standard output from one process to standard input of the next. Read these characters exactly because changing one symbol may change the result or overwrite a file.

Linux commands produce output on standard output and errors on standard error. Redirection operators send those streams to files. Pipes send the output of one command directly into another command.

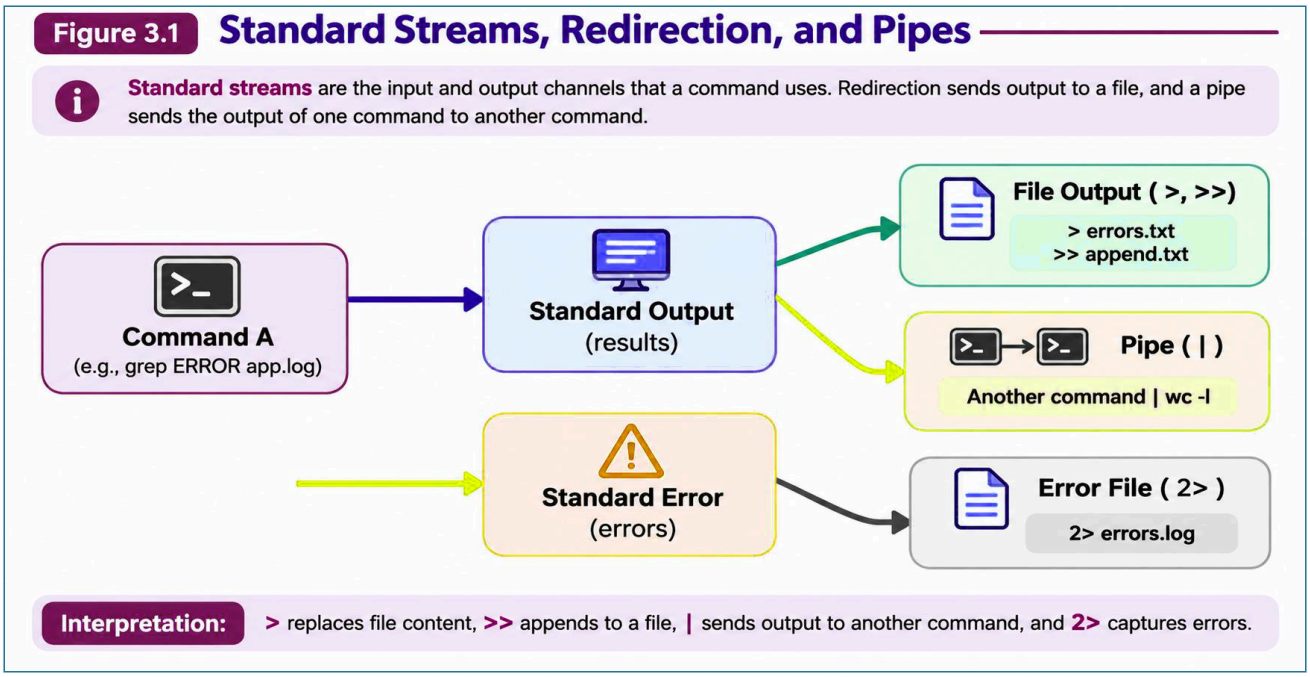


Figure 3.1 Standard output, standard error, redirection, and pipes.

```
grep "ERROR" app.log > errors.txt
grep "ERROR" app.log >> errors.txt
grep "ERROR" app.log | wc -l
```

3.4 Text-processing commands

3.4.1 sort

Definition

The sort command arranges lines of text into a chosen order. By default it uses text collation, which may differ from numerical order. Use -n for numbers, -r to reverse the order, and -u to keep one copy of each identical sorted line.

Why students use this command

Use it to order names, scores and structured reports.

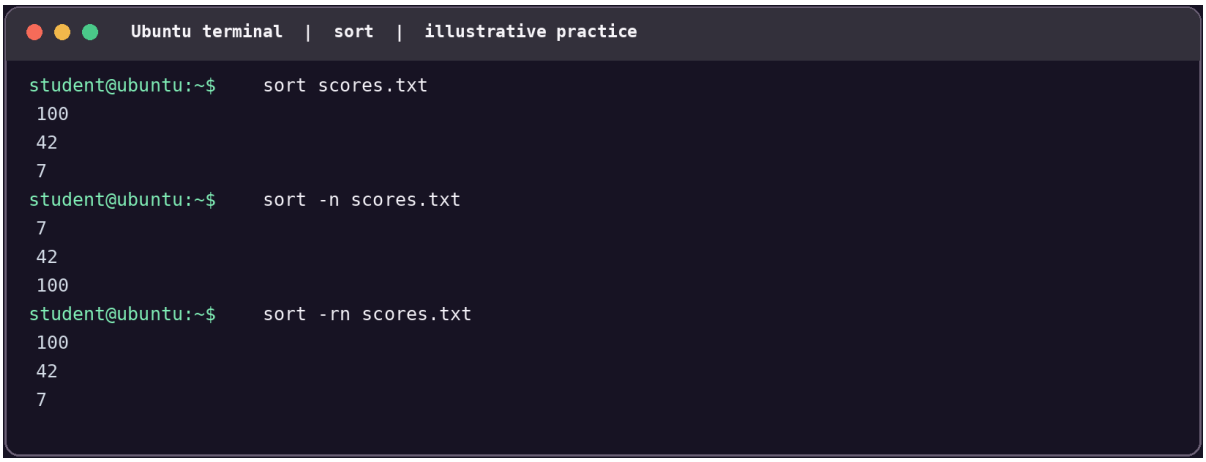
Syntax

```
sort [options] file
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-n`	Compare values numerically	Function: compare values numerically. Example: sort -n scores.txt
`-r`	Reverse the sort order	Function: reverse the sort order. Example: sort -r names.txt
`-u`	Return unique sorted lines	Function: return unique sorted lines. Example: sort -u cities.txt

Practice screenshot — test commands and example output



3.4.2 uniq

Definition

The uniq command removes, counts, or selects repeated lines that appear next to one another. It does not automatically find duplicates separated by other lines. For this reason, students often use sort first and then pipe its output into uniq or uniq -c.

Why students use this command

Use it with sort to produce distinct categories or frequency counts.

Syntax

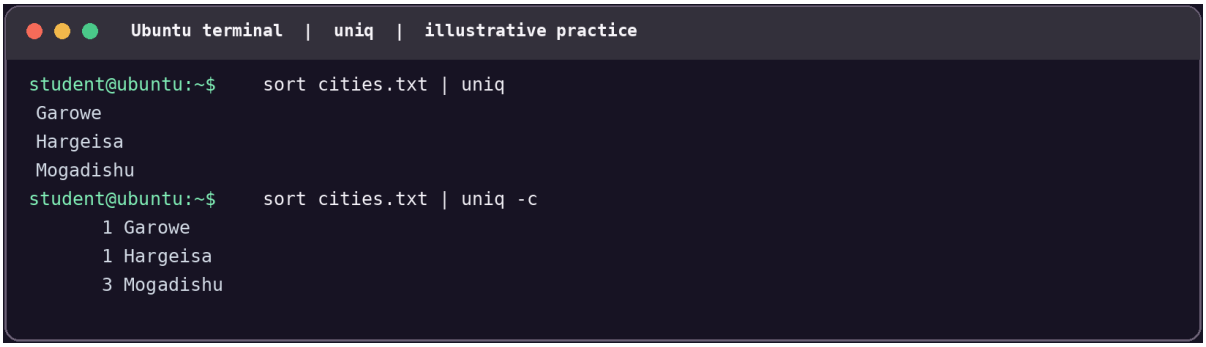
```
uniq [options] [file]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-c`	Prefix each group with its frequency	Function: prefix each group with its frequency. Example: uniq -c sorted.txt

Option / Shortcut	Name	Description & Example Usage
-d	Display only repeated groups	Function: display only repeated groups. Example: uniq -d sorted.txt
-u	Display only lines that occur once	Function: display only lines that occur once. Example: uniq -u sorted.txt

Practice screenshot — test commands and example output



3.4.3 cut

Definition

The cut command extracts selected characters or fields from every line of input. A field is a piece of text separated from neighbouring pieces by a character such as a colon or comma. Use -d to define the separator and -f to choose the field number. It is suited to simple delimited files, not every form of complex CSV.

Why students use this command

Use it with simple structured files, such as colon-separated records.

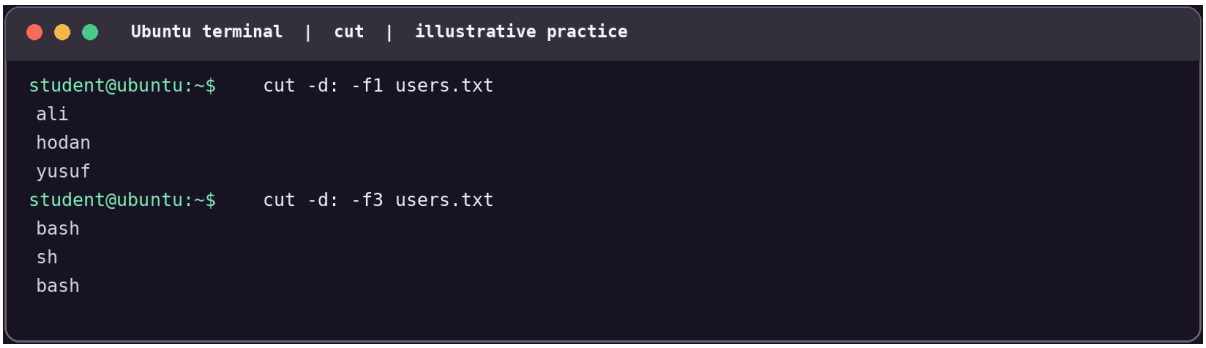
Syntax

```
cut [options] file
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-d CHAR	Choose the field separator	Function: choose the field separator. Example: cut -d: -f1 users.txt
-f N	Select the Nth field or comma-separated set of fields	Function: select the Nth field or comma-separated set of fields. Example: cut -d: -f1,3 users.txt
-c RANGE	Select character positions	Function: select character positions. Example: cut -c1-4 notes.txt

Practice screenshot — test commands and example output



Important note. cut is not a complete CSV parser for quoted fields containing commas.

3.4.4 paste

Definition

The paste command joins corresponding lines from two or more files side by side. By default it places a tab between values, producing a simple table. The -d option can choose a different separator, such as a comma. It is useful when separate files contain matching rows of information.

Why students use this command

Use it for simple side-by-side text tables.

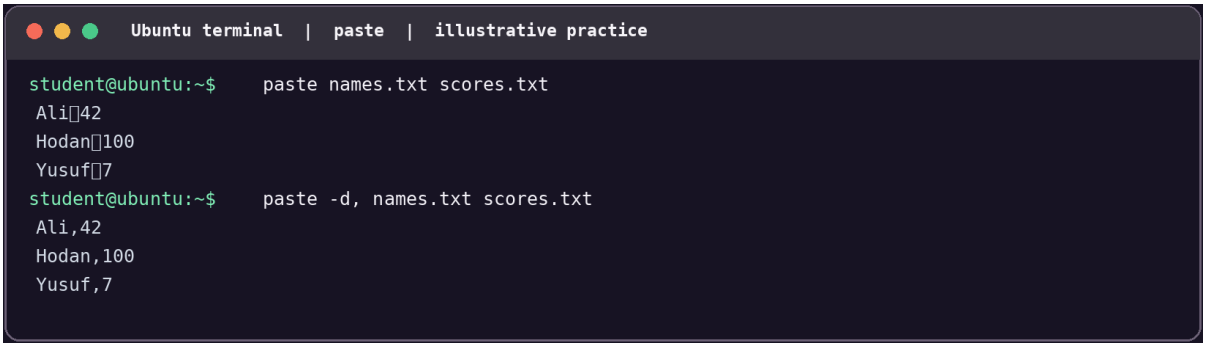
Syntax

```
paste [options] FILE1 FILE2
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-d CHAR	Set a different output separator	Function: set a different output separator. Example: paste -d, names.txt scores.txt
-s	Join lines serially rather than row by row	Function: join lines serially rather than row by row. Example: paste -s -d, names.txt

Practice screenshot — test commands and example output



3.4.5 wc

Definition

The wc (word count) command counts lines, words, bytes, or characters in a file or text stream. Use -l for lines, -w for words, -c for bytes, and -m for characters. It is especially useful in a pipeline, such as grep ERROR app.log | wc -l to count matching lines.

Why students use this command

Use it to count records, words in a document or the volume of program output.

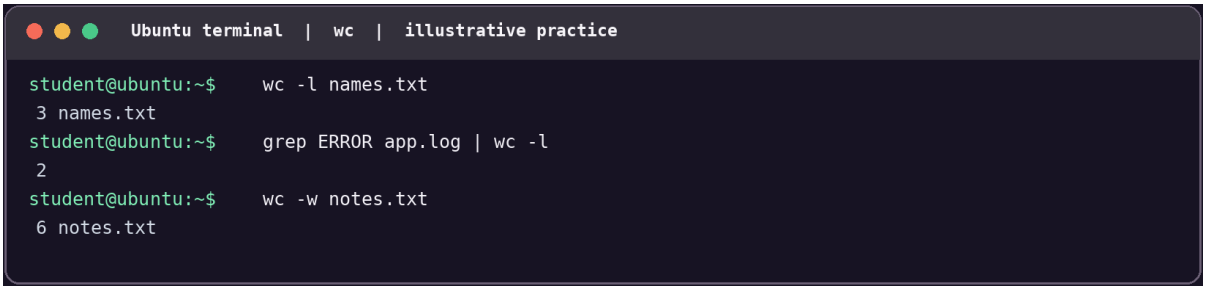
Syntax

```
wc [options] file
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-l	Count newline characters / lines in normal line-oriented files	Function: count newline characters / lines in normal line-oriented files. Example: wc -l names.txt
-w	Count whitespace-delimited words	Function: count whitespace-delimited words. Example: wc -w notes.txt
-c	Count bytes	Function: count bytes. Example: wc -c notes.txt
-m	Count characters	Function: count characters. Example: wc -m notes.txt

Practice screenshot — test commands and example output



3.4.6 tr

Definition

The tr (translate) command replaces, deletes, or reduces repeated characters from text it receives on standard input. For example, tr a-z A-Z converts lowercase letters into uppercase letters. It normally reads from a pipe or input redirection instead of from a filename argument.

Why students use this command

Use it for simple character-level cleaning or case conversion.

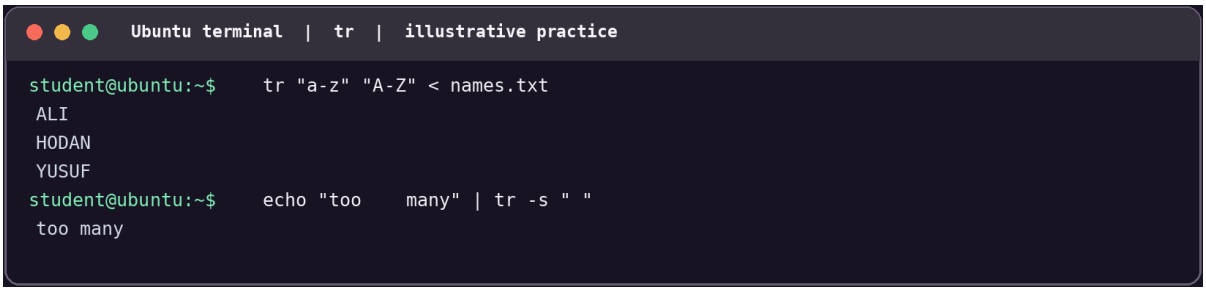
Syntax

```
tr [options] SET1 [SET2]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-d	Delete characters in the chosen set	Function: delete characters in the chosen set. Example: tr -d "0-9"
-s	Squeeze repeated characters in the chosen set	Function: squeeze repeated characters in the chosen set. Example: tr -s " "

Practice screenshot — test commands and example output



3.5 Archives and compression

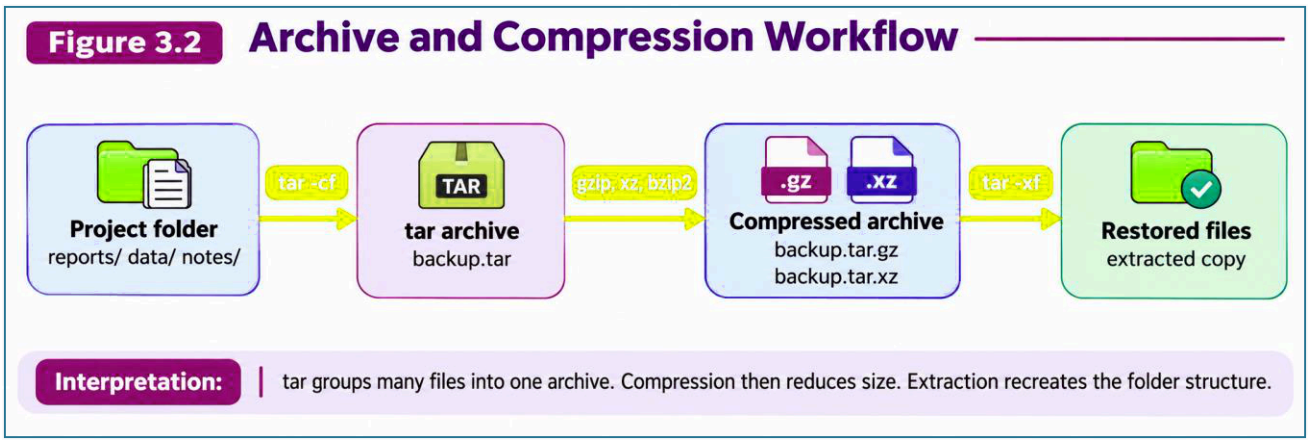


Figure 3.2 Archive creation, compression, and extraction.

3.5.1 tar

Definition

The tar (tape archive) command groups files and directories into a single archive while preserving their relative paths. It can create an archive, list its contents, or extract stored files. A .tar file is not necessarily compressed; options such as -z, -j, and -J request gzip, bzip2, or xz compression.

Why students use this command

Use it for Linux backups and distribution of directory trees.

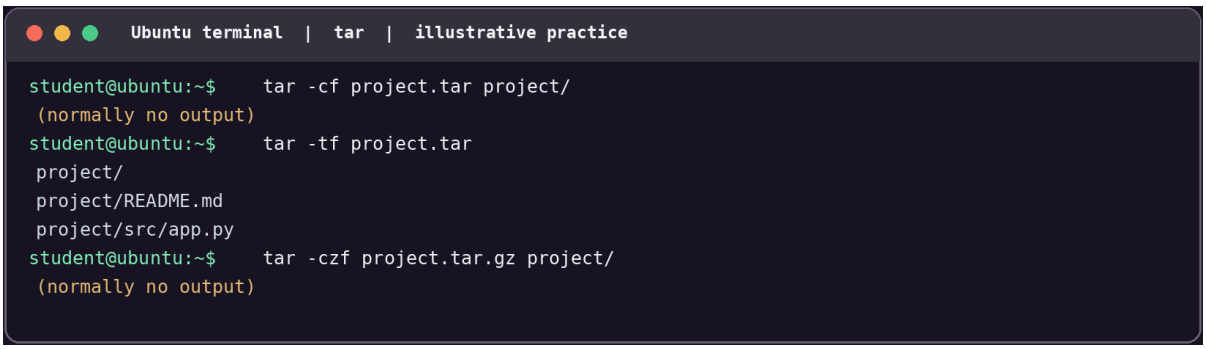
Syntax

```
tar [options] ARCHIVE SOURCES...
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-c`	Create archive	Function: create archive. Example: tar -cf backup.tar project/
`-x`	Extract archive	Function: extract archive. Example: tar -xf backup.tar
`-t`	List archive members	Function: list archive members. Example: tar -tf backup.tar
`-f FILE`	Specify archive filename	Function: specify archive filename. Example: tar -cf backup.tar data/
`-v`	Show processed paths	Function: show processed paths. Example: tar -cvf backup.tar data/
`-z`	Use gzip compression	Function: use gzip compression. Example: tar -czf backup.tar.gz project/
`-j`	Use bzip2 compression	Function: use bzip2 compression. Example: tar -cjf backup.tar.bz2 project/
`-J`	Use xz compression	Function: use xz compression. Example: tar -cJf backup.tar.xz project/

Practice screenshot — test commands and example output



Important note. Check member paths before extracting a file from an unknown source.

3.5.2 gzip

Definition

The gzip command compresses a file into the .gz format to reduce storage space. By default, it commonly replaces the original file with a compressed version. Use -k when you want to keep the original. For multiple files and folders, gzip is often combined with tar.

Why students use this command

Use it to reduce size of large text files; use tar when many files must be kept together.

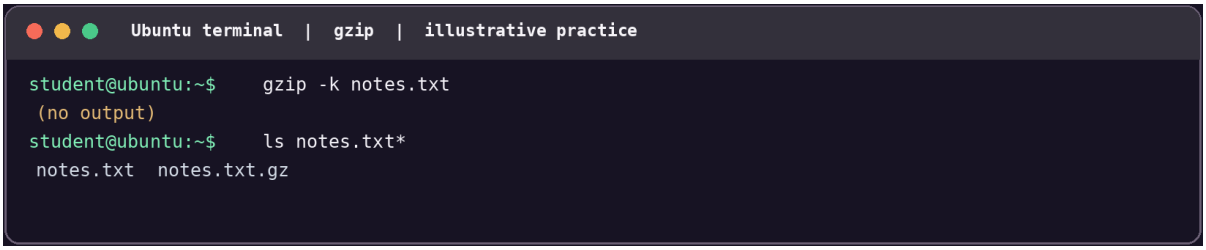
Syntax

```
gzip [options] FILE
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-k	Keep the original file	Function: keep the original file. Example: gzip -k notes.txt
-9	Use the strongest gzip compression level (often slower)	Function: use the strongest gzip compression level (often slower). Example: gzip -9 notes.txt

Practice screenshot — test commands and example output



3.5.3 gunzip

Definition

The gunzip command decompresses a .gz file and restores its uncompressed contents. It normally removes the compressed input after successful restoration. With -c it writes the uncompressed data to standard output without replacing the compressed file.

Why students use this command

Use it to recover a gzip-compressed file.

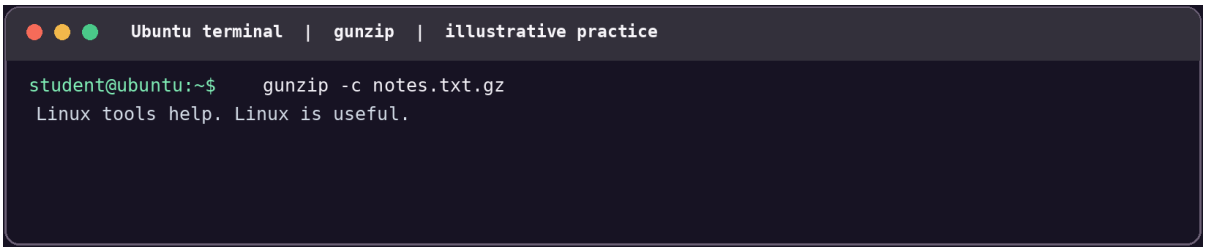
Syntax

```
gunzip [options] FILE.gz
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-c	Write decompressed content to standard output	Function: write decompressed content to standard output. Example: gunzip -c notes.txt.gz

Practice screenshot — test commands and example output



3.5.4 bzip2

Definition

The bzip2 command compresses a file using the bzip2 format, usually producing a .bz2 extension. It is one of several Linux compression methods. Use -k to preserve the original file or -d to reverse compression when appropriate.

Why students use this command

Use it when a .bz2 format is required for a workflow.

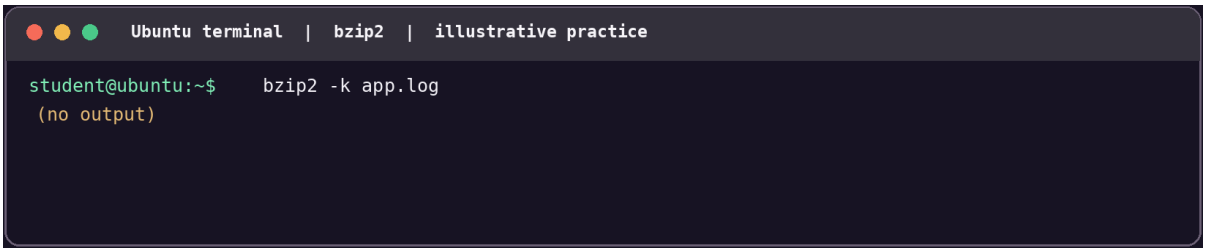
Syntax

```
bzip2 [options] FILE
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-k	Keep the original input file	Function: keep the original input file. Example: bzip2 -k app.log
-d	Decompress a .bz2 file	Function: decompress a .bz2 file. Example: bzip2 -d app.log.bz2

Practice screenshot — test commands and example output



3.5.5 bunzip2

Definition

The bunzip2 command restores files compressed with bzip2. For example, bunzip2 report.txt.bz2 usually produces report.txt. It acts on a compressed file rather than directly managing a folder structure.

Why students use this command

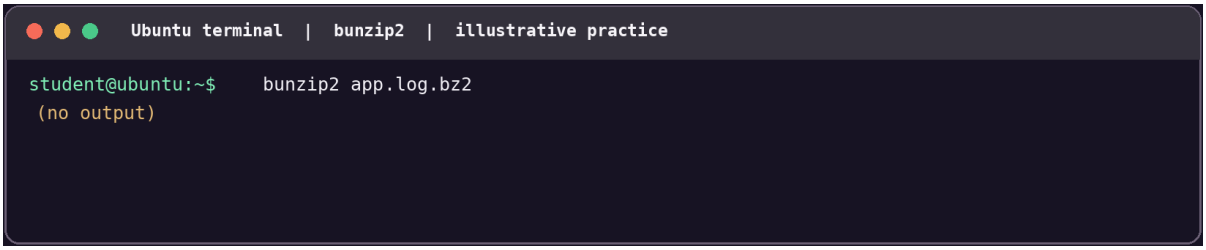
Use it to restore .bz2 data.

Syntax

```
bunzip2 FILE.bz2
```

Options: The examples below use the command without additional flags.

Practice screenshot — test commands and example output



3.5.6 xz

Definition

The xz command compresses a file using the xz format, usually producing a .xz extension. It often achieves strong compression but may take more processing time than gzip. Use -k to keep the original or -d to decompress.

Why students use this command

Use it for distributing software or archives where space reduction matters.

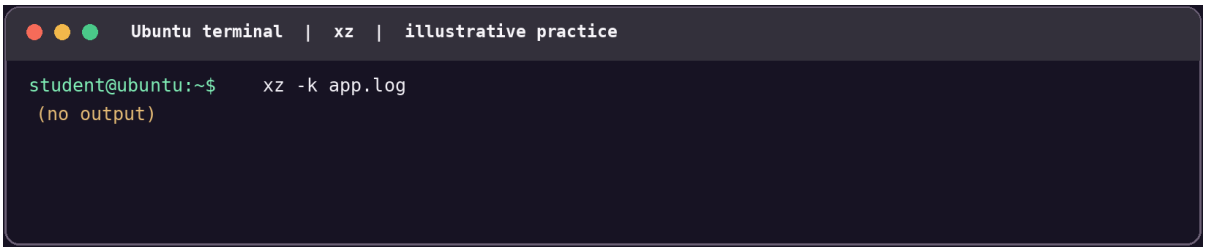
Syntax

```
xz [options] FILE
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-k	Keep original input file	Function: keep original input file. Example: xz -k app.log
-d	Decompress an .xz file	Function: decompress an .xz file. Example: xz -d app.log.xz

Practice screenshot — test commands and example output



3.5.7 unxz

Definition

The unxz command decompresses a file created with xz. A file named report.txt.xz is typically restored as report.txt. It is equivalent to using xz -d for the same purpose.

Why students use this command

Use it to restore .xz files.

Syntax

```
unxz FILE.xz
```

Options: The examples below use the command without additional flags.

Practice screenshot — test commands and example output



3.5.8 zip

Definition

The zip command creates a ZIP archive containing files and, with -r, folders and their contents. ZIP files are widely supported on Linux, Windows, and macOS, making them suitable for sharing coursework or project folders.

Why students use this command

Use it when sharing collections of files across different operating systems.

Syntax

```
zip [options] ARCHIVE.zip SOURCES...
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-r	Include directories recursively	Function: include directories recursively. Example: zip -r project.zip project/
-9	Use maximum available compression level	Function: use maximum available compression level. Example: zip -9 data.zip report.txt

Practice screenshot — test commands and example output



3.5.9 unzip

Definition

The unzip command lists or extracts files from ZIP archives. Use -l to view the stored filenames before extraction and -d to choose a destination folder. Checking an archive's contents first helps avoid unexpected files being created in the current directory.

Why students use this command

Use it to inspect and restore ZIP packages.

Syntax

```
unzip [options] ARCHIVE.zip
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-l`	List contents without extracting	Function: list contents without extracting. Example: unzip -l project.zip
`-d DIR`	Extract into a chosen destination directory	Function: extract into a chosen destination directory. Example: unzip project.zip -d restored/

Practice screenshot — test commands and example output



3.6 Linux text editors

A text editor changes the contents of a file. Unlike cat or less, an editor can save modifications. Start with a harmless file in your home directory, not an important system configuration file.

Nano is designed for straightforward typing with visible keyboard shortcuts. Vim uses separate editing modes. Both work inside a terminal and are helpful when a graphical desktop is unavailable.

3.6.1 nano

Definition

The nano command opens a simple terminal text editor. You can begin typing immediately, and the shortcut hints at the bottom of the screen show how to save, search, and exit. Press Ctrl+O to save, Enter to confirm the filename, and Ctrl+X to leave the editor.

Why students use this command

Use Nano to create or change simple text documents and sample configuration files.

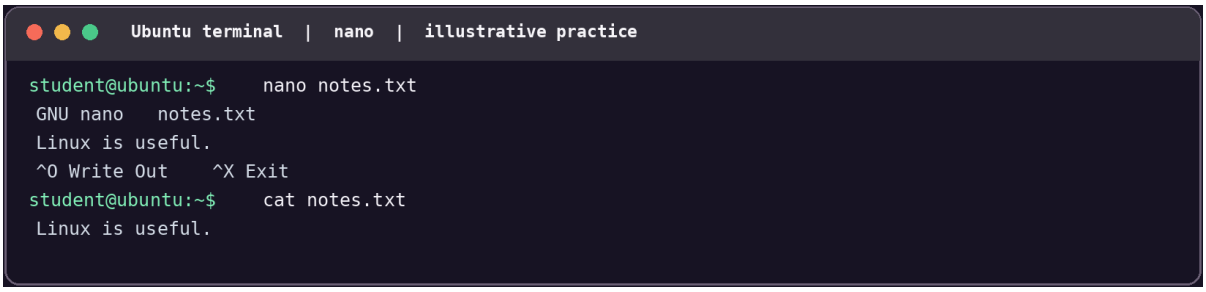
Syntax

```
nano [filename]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
Ctrl+O	Save the file; confirm the filename with Enter	Function: save the file; confirm the filename with Enter. Example: Ctrl+O
Ctrl+X	Exit Nano; prompts if changes are unsaved	Function: exit Nano; prompts if changes are unsaved. Example: Ctrl+X
Ctrl+W	Search for text	Function: search for text. Example: Ctrl+W
Ctrl+K	Cut current line	Function: cut current line. Example: Ctrl+K
Ctrl+U	Paste cut text	Function: paste cut text. Example: Ctrl+U

Practice screenshot — test commands and example output



Important note. Edit only your own practice files. Save with Ctrl+O, Enter, then exit with Ctrl+X.

3.6.2 vim

Definition

The vim command opens Vim, a text editor that works through different modes. In Normal mode, keys run editing actions. Press i to enter Insert mode for typing, then Esc to return to Normal mode. Use :w to save, :q to quit, or :wq to save and quit.

Why students use this command

Use Vim when you are ready for keyboard-controlled editing, including remote servers.

Syntax

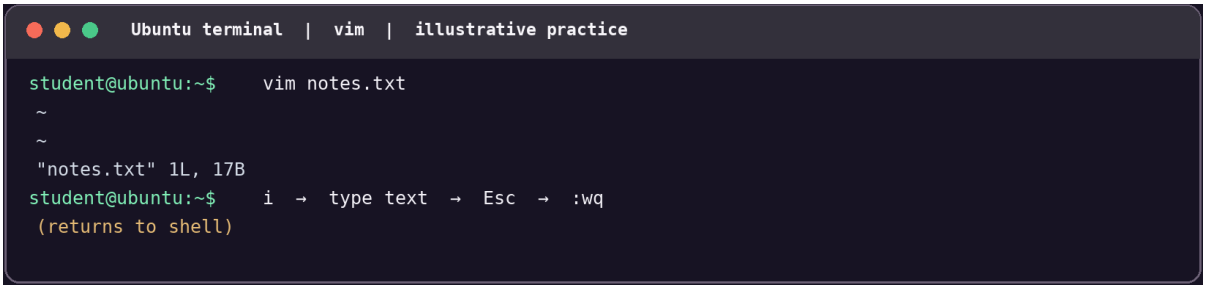
```
vim [filename]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
i	Enter Insert mode from Normal mode	Function: enter Insert mode from Normal mode. Example: i

Esc	Return to Normal mode	Function: return to Normal mode. Example: Esc
:w	Write/save file from command-line mode	Function: write/save file from command-line mode. Example: :w
:q	Quit if there are no unsaved changes	Function: quit if there are no unsaved changes. Example: :q
:wq	Save and quit	Function: save and quit. Example: :wq
:q!	Quit without saving changes	Function: quit without saving changes. Example: :q!
dd	Delete current line in Normal mode	Function: delete current line in Normal mode. Example: dd
yy	Copy current line in Normal mode	Function: copy current line in Normal mode. Example: yy
p	Paste after cursor in Normal mode	Function: paste after cursor in Normal mode. Example: p

Practice screenshot — test commands and example output



Important note. Do not type :wq while still in Insert mode; press Esc first.

3.7 Hard links and symbolic links

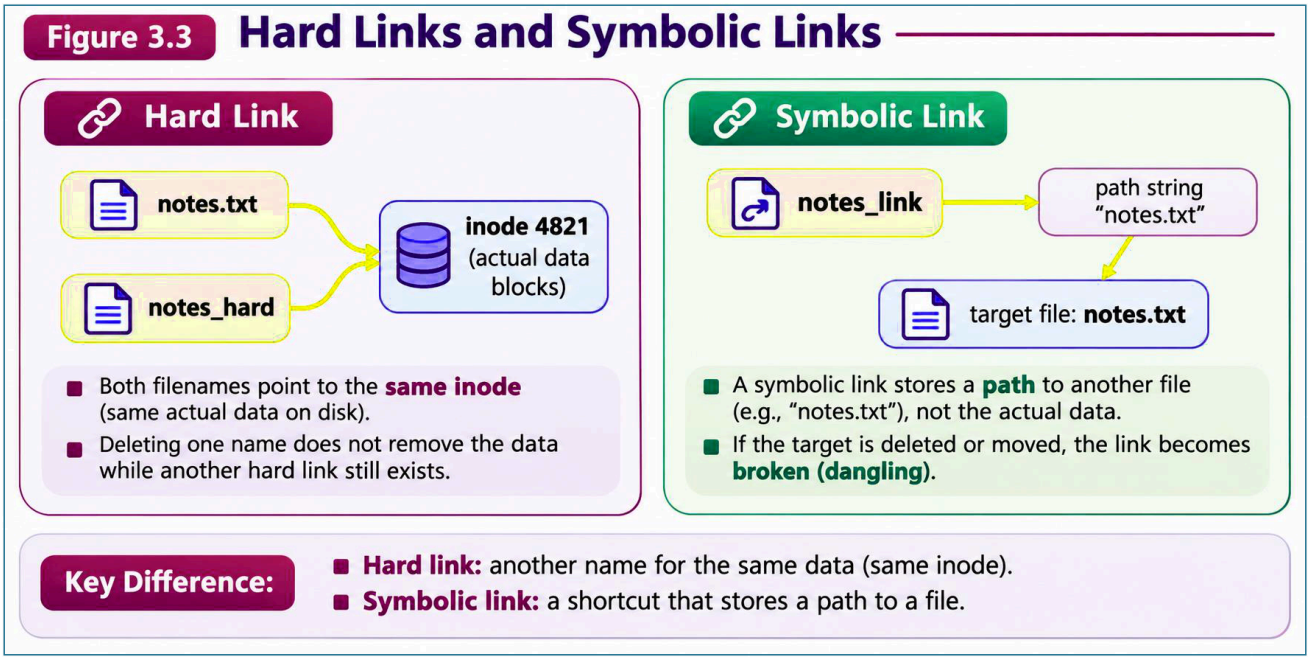


Figure 3.3 Hard links and symbolic links.

3.7.1 ln

Definition

The ln (link) command creates an additional directory entry referring to an existing file. Without -s, it creates a hard link, which shares the same underlying inode and data as the original filename. Hard links usually cannot cross filesystems.

Why students use this command

Use it when two names should refer to the same file within one filesystem.

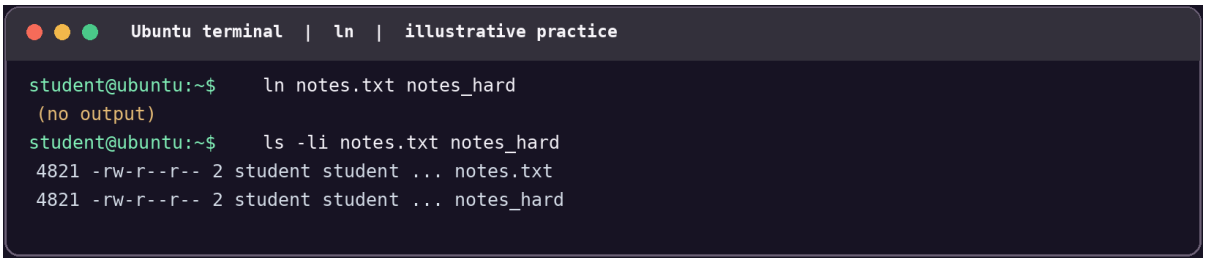
Syntax

```
ln TARGET LINKNAME
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-v	Describe each link created	Function: describe each link created. Example: ln -v notes.txt notes_hard

Practice screenshot — test commands and example output



3.7.2 ln -s

Definition

The ln -s command creates a symbolic link, often called a symlink. A symbolic link stores the path to another file or directory rather than sharing the same data inode. It is useful for shortcuts and references, but it breaks if the target path no longer exists.

Why students use this command

Use it for shortcuts and references to files in other directories or filesystems.

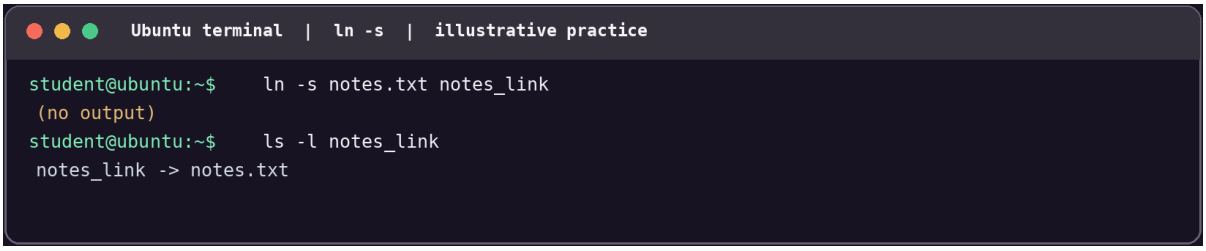
Syntax

```
ln -s TARGET LINKNAME
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-s	Create a symbolic link rather than a hard link	Function: create a symbolic link rather than a hard link. Example: ln -s notes.txt notes_link

Practice screenshot — test commands and example output



Important note. If its target is moved or removed, a symbolic link can become broken.

CHAPTER 4 — Process, Service, and System Management

Chapter overview

This chapter explains how Linux runs programs as processes, how background jobs are controlled, how signals are used to stop or reload programs, how system information is read, and how services are managed with systemd. It also introduces logging and first-line troubleshooting.

Learning objectives

- Explain PID, PPID, foreground jobs, and background jobs.
- Use ps, top, htop, jobs, bg, and fg correctly.
- Stop processes safely with kill, pkill, or killall.
- Interpret nice values and change process priority with nice and renice.
- Use uname, uptime, free, lscpu, lsblk, /proc, and /sys to describe the system.
- Manage services and logs with systemctl, journalctl, dmesg, and shutdown commands.

4.1 Processes and job control

A program is an executable set of instructions stored on disk, while a process is one running instance of those instructions. A process has its own identifiers, memory, resources, and status. Several instances of the same program may run simultaneously, each with a different PID.

Foreground and background are terms for how a job uses a terminal, not for whether it consumes CPU. A background process can use considerable CPU time, while a foreground process can spend most of its time waiting for input.

A process is a running program. Each process has a PID (process ID). The process that started it is the parent process, identified by the PPID (parent process ID).

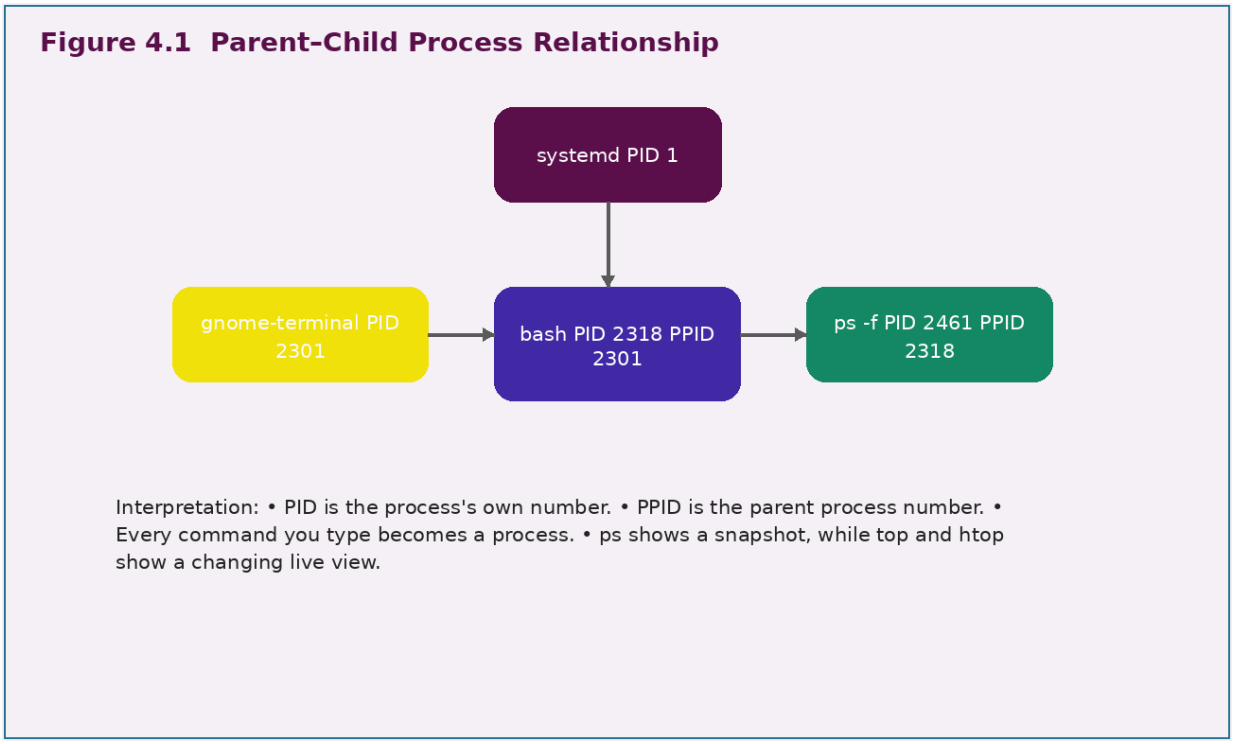


Figure 4.1 A simple parent–child process relationship.

4.1.1 ps

Definition

The ps (process status) command prints a snapshot of running processes. A process is a program currently executing, and each has a process ID (PID). With options, ps can show the process owner, parent PID, CPU use, and state. Its output does not refresh continuously.

Why students use this command

Use it to inspect running programs and identify a process before changing it.

Syntax

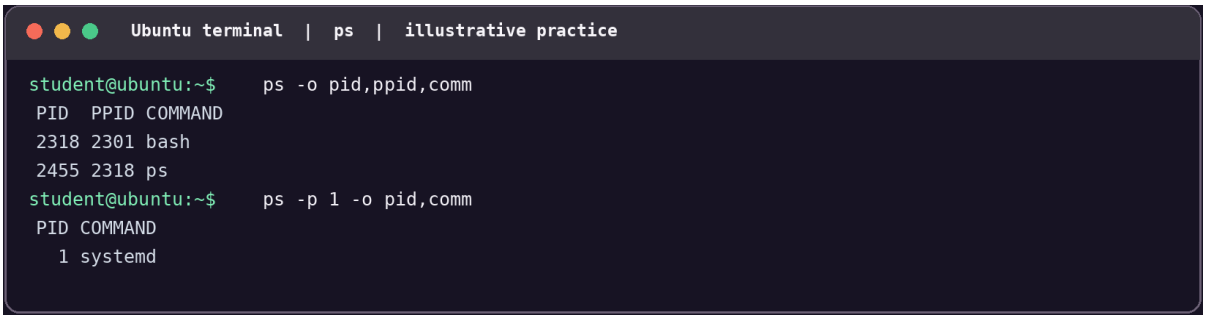
```
ps [options]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-f`	Full listing with owner and PPID	Function: full listing with owner and PPID. Example: ps -f
`-e`	Include all processes	Function: include all processes. Example: ps -e
`aux`	BSD-style view of all processes and resource use	Function: bSD-style view of all processes and resource use. Example: ps aux

Option / Shortcut	Name	Description & Example Usage
-p PID	Select one PID	Function: select one PID. Example: ps -p 1
-o COLUMNS	Select columns to display	Function: select columns to display. Example: ps -o pid,ppid,comm

Practice screenshot — test commands and example output



4.1.2 top

Definition

The top command opens a live text-based display of running processes and system resource use. It updates regularly so you can see which programs consume CPU or memory. Inside top, press P to sort by CPU, M by memory, and q to exit.

Why students use this command

Use it when checking which process is making a machine slow.

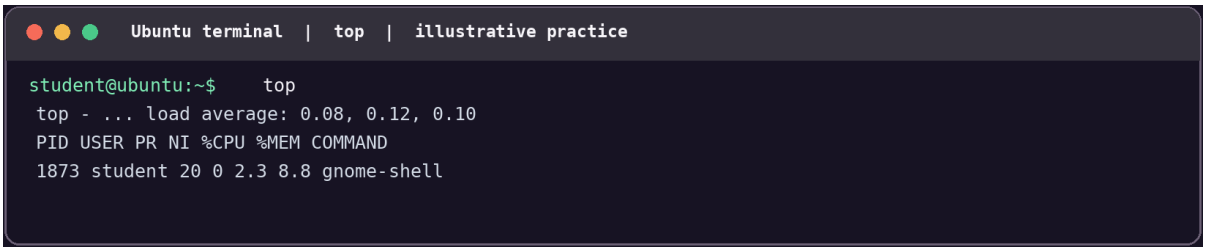
Syntax

```
top [options]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-u USER	Display processes belonging to one user	Function: display processes belonging to one user. Example: top -u student
-p PID	Show a particular process	Function: show a particular process. Example: top -p 2318

Practice screenshot — test commands and example output



4.1.3 htop

Definition

The htop command is an interactive process monitor similar to top, with a clearer display, coloured resource bars, and convenient keyboard controls. It shows process activity, CPU use, and memory use. It may need installation before use on some Ubuntu systems.

Why students use this command

Use it when you want a more visual view than top.

Syntax

```
htop [options]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-u USER	Show one user's processes	Function: show one user's processes. Example: htop -u student
-t	Start with tree view	Function: start with tree view. Example: htop -t

Practice screenshot — test commands and example output



4.1.4 jobs

Definition

The jobs command lists tasks started or suspended in the current shell session. A job has a shell job number such as %1, which differs from its system process ID. It helps you see which commands are running in the background and which are stopped.

Why students use this command

Use it after starting commands with & or suspending them with Ctrl+Z.

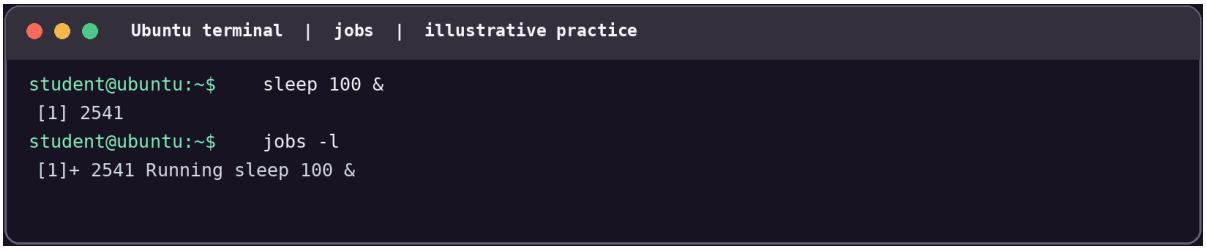
Syntax

```
jobs [options]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-l	Include the process PID in the listing	Function: include the process PID in the listing. Example: jobs -l

Practice screenshot — test commands and example output



4.1.5 bg

Definition

The bg (background) command resumes a stopped shell job and lets it continue without occupying the foreground of your terminal. Use jobs to find its job number, then bg %1 to resume job 1. It only controls jobs belonging to the current shell.

Why students use this command

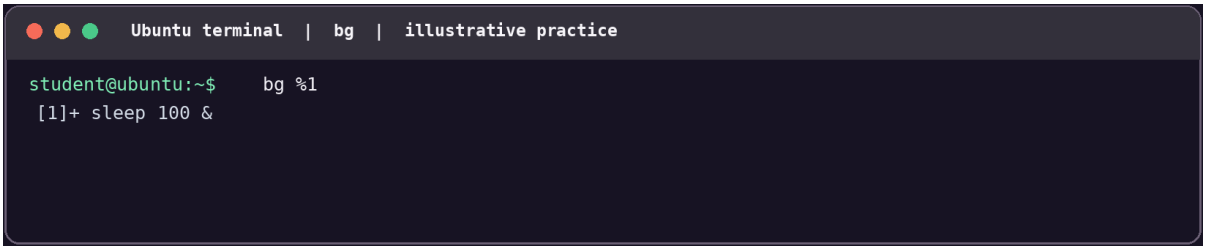
Use it after Ctrl+Z when the paused job should continue without holding the terminal.

Syntax

```
bg [%JOB]
```

Options: The examples below use the command without additional flags.

Practice screenshot — test commands and example output



4.1.6 fg

Definition

The fg (foreground) command brings a shell job into the foreground, so it interacts with your terminal again. A common sequence is Ctrl+Z to pause a command, jobs to see its number, and fg %1 to continue it in front.

Why students use this command

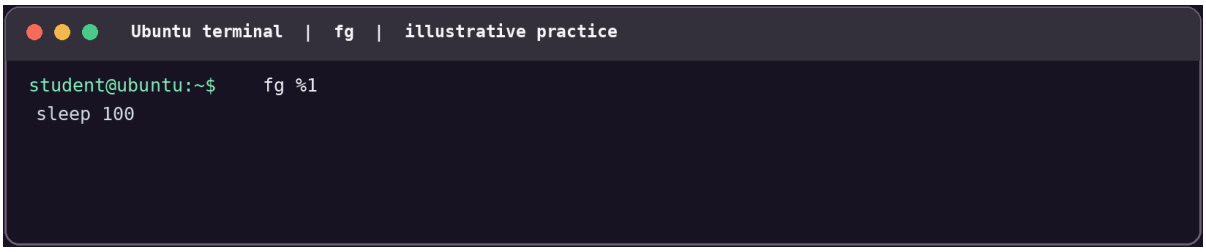
Use it when a job must receive keyboard input again.

Syntax

`fg [%JOB]`

Options: The examples below use the command without additional flags.

Practice screenshot — test commands and example output



4.2 Signals and process priority

4.2.1 kill

Definition

The kill command sends a signal to a process identified by its PID. Despite the name, it does not always force immediate termination: the default SIGTERM politely requests that the process end. SIGKILL (-9) stops a process immediately and should be a last resort.

Why students use this command

Use it to stop your own practice processes after checking the PID.

Syntax

`kill [-SIGNAL] PID`

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-TERM`	Request graceful termination (default)	Function: request graceful termination (default). Example: kill -TERM 3120
`-KILL or -9`	Force immediate termination; no cleanup	Function: force immediate termination; no cleanup. Example: kill -9 3120
`-HUP`	Hang-up signal; many services treat it as a reload request	Function: hang-up signal; many services treat it as a reload request. Example: kill -HUP 3120
`-l`	List recognised signal names	Function: list recognised signal names. Example: kill -l

Practice screenshot — test commands and example output



Important note. Use SIGKILL only if the process did not respond to SIGTERM.

4.2.2 pgrep

Definition

The pgrep command searches running processes by name or another selection rule and prints their PIDs. It helps you identify the correct process before using kill. The -a option also displays the matching command line.

Why students use this command

Use it to identify matches before sending name-based signals.

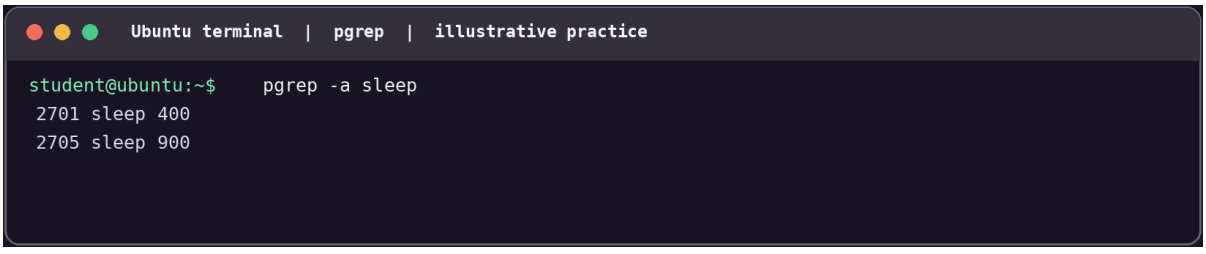
Syntax

```
pgrep [options] PATTERN
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-a	Show matching PIDs and command lines	Function: show matching PIDs and command lines. Example: pgrep -a sleep
-u USER	Limit matches to a user	Function: limit matches to a user. Example: pgrep -u student sleep

Practice screenshot — test commands and example output



4.2.3 pkill

Definition

The pkill command sends a signal to running processes that match a name or selection rule. It combines process searching and signalling, so it may affect more than one process. Check matches with pgrep before using it.

Why students use this command

Use it cautiously after inspecting the matches with pgrep.

Syntax

```
pkill [options] PATTERN
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-u USER	Match only processes owned by USER	Function: match only processes owned by USER. Example: pkill -u student sleep
-TERM	Send SIGTERM explicitly	Function: send SIGTERM explicitly. Example: pkill -TERM sleep

Practice screenshot — test commands and example output



4.2.4 killall

Definition

The killall command sends a signal to processes with a specified executable name. On Ubuntu, it commonly targets every matching process name, so confirm the name and number of matches before running it. Like kill, it normally sends SIGTERM by default.

Why students use this command

Use it only after confirming that every process with that name should receive the signal.

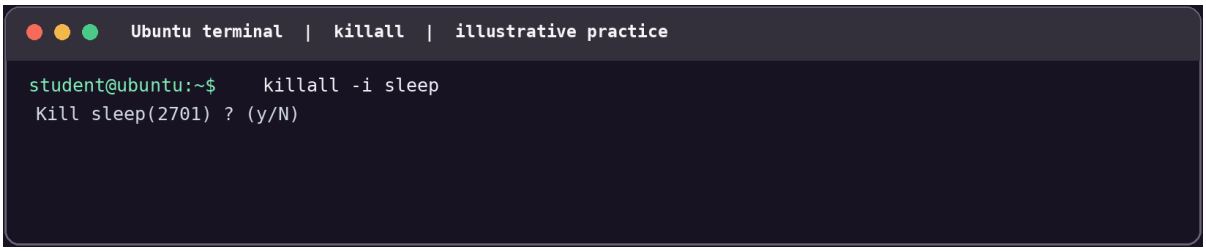
Syntax

```
killall [options] NAME
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-i	Ask before killing each matched process	Function: ask before killing each matched process. Example: killall -i sleep
-u USER	Limit to processes owned by USER	Function: limit to processes owned by USER. Example: killall -u student sleep

Practice screenshot — test commands and example output



4.2.5 nice

Definition

The nice command starts a program with a selected niceness value that influences CPU scheduling priority. Niceness commonly ranges from -20 to 19; a larger value means the process is less favoured when competing for CPU time. Normal users can usually raise niceness on their own processes.

Why students use this command

Use it to start long nonurgent jobs with less scheduling priority.

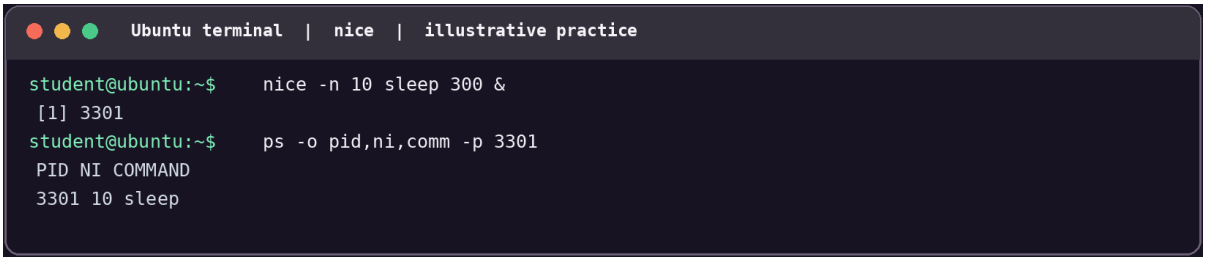
Syntax

```
nice [-n VALUE] COMMAND
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-n VALUE	Set niceness, generally between -20 and 19	Function: set niceness, generally between -20 and 19. Example: nice -n 10 sleep 300

Practice screenshot — test commands and example output



4.2.6 renice

Definition

The renice command changes the niceness value of an already-running process. Give it a PID and a new niceness value to adjust how the scheduler treats the process during CPU competition. Lowering niceness generally requires administrator privileges.

Why students use this command

Use it to adjust an existing job when resource priorities change.

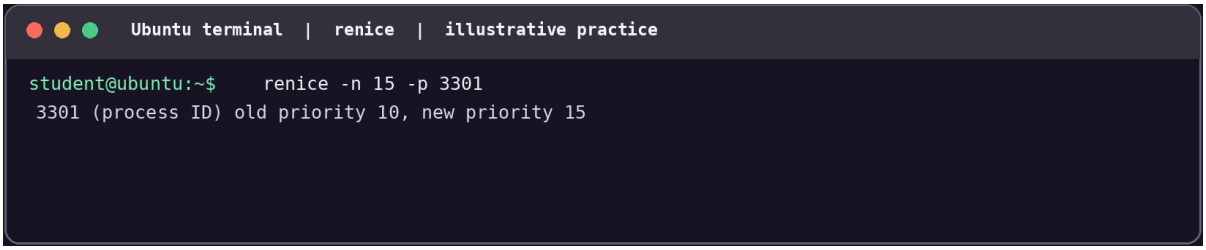
Syntax

```
renice -n VALUE -p PID
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-n VALUE`	New niceness value	Function: new niceness value. Example: renice -n 15 -p 3301
`-p PID`	Choose a process by PID	Function: choose a process by PID. Example: renice -n 15 -p 3301

Practice screenshot — test commands and example output



4.3 System information

4.3.1 uname

Definition

The uname (Unix name) command reports information about the running Linux kernel and machine architecture. It can show the kernel release, hostname, processor architecture, or a combined summary. It helps students identify which kernel is actually running.

Why students use this command

Use it to document which kernel you are using.

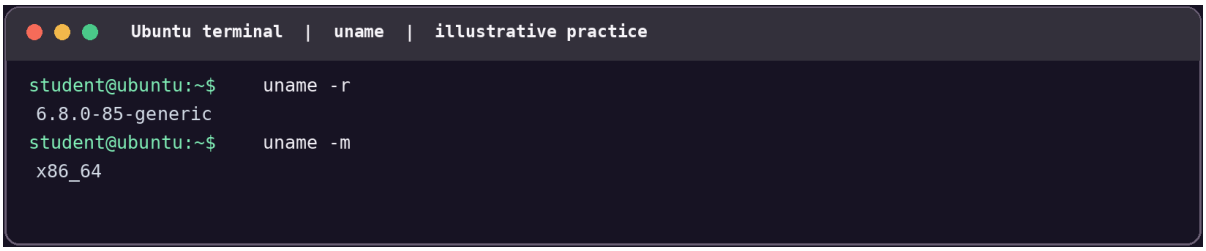
Syntax

```
uname [options]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-r`	Kernel release	Function: kernel release. Example: uname -r
`-m`	Hardware architecture	Function: hardware architecture. Example: uname -m
`-n`	Network node name	Function: network node name. Example: uname -n
`-a`	All common information	Function: all common information. Example: uname -a

Practice screenshot — test commands and example output



4.3.2 hostnamectl

Definition

The hostnamectl command displays the computer's hostname and other system identity information, often including the operating-system version, kernel, and virtualization environment. Its read-only output is useful when documenting a laboratory VM.

Why students use this command

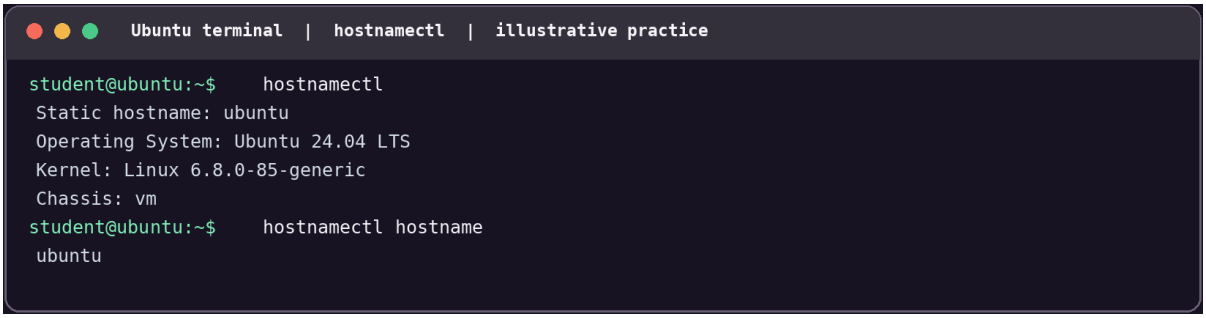
Use it to identify the system before troubleshooting.

Syntax

```
hostnamectl [command]
```

Options: The examples below use the command without additional flags.

Practice screenshot — test commands and example output



4.3.3 uptime

Definition

The uptime command reports how long the system has been running since its last boot. It also displays logged-in user count and three load averages for approximately the previous 1, 5, and 15 minutes. Compare load averages with available CPU count to interpret system activity.

Why students use this command

Use it as a first check when a machine seems overloaded or unexpectedly rebooted.

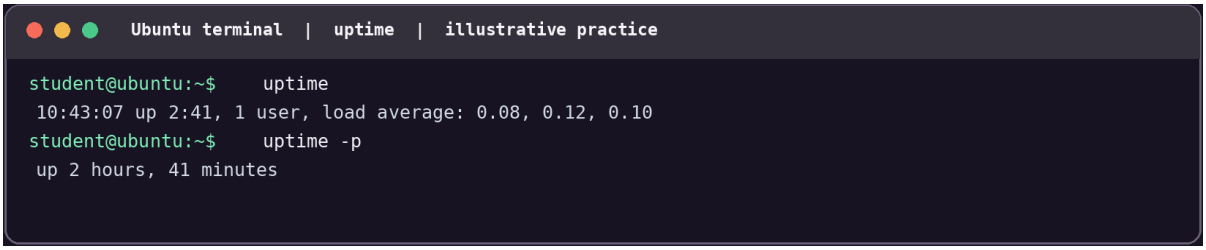
Syntax

```
uptime [options]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-p`	Print uptime in friendly words	Function: print uptime in friendly words. Example: uptime -p
`-s`	Print the boot time	Function: print the boot time. Example: uptime -s

Practice screenshot — test commands and example output



4.3.4 free

Definition

The free command reports how the computer uses RAM and swap memory. It shows total, used, free, and available amounts. The available column is often more useful than free because Linux can reuse cached memory when applications need it.

Why students use this command

Use it when diagnosing memory pressure.

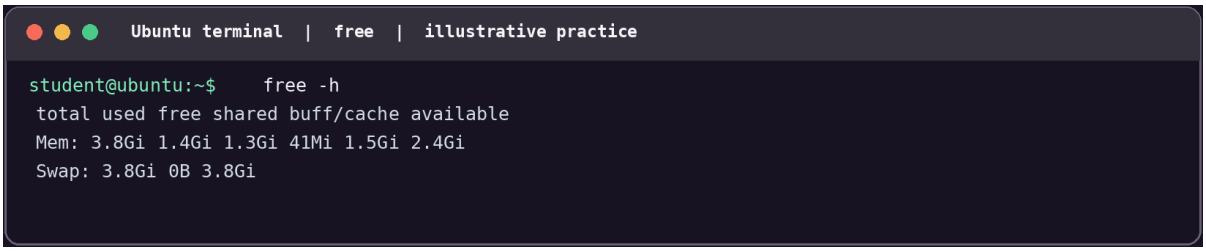
Syntax

```
free [options]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-h`	Human-readable units	Function: human-readable units. Example: free -h
`-m`	Show values in MiB	Function: show values in MiB. Example: free -m
`-s N`	Update every N seconds	Function: update every N seconds. Example: free -s 2

Practice screenshot — test commands and example output



4.3.5 lscpu

Definition

The lscpu command displays information about the processor and CPU architecture, such as model name, logical CPU count, and virtualization details. It is useful for understanding the hardware resources assigned to an Ubuntu virtual machine.

Why students use this command

Use it when documenting CPUs or checking a VM's virtual processor allocation.

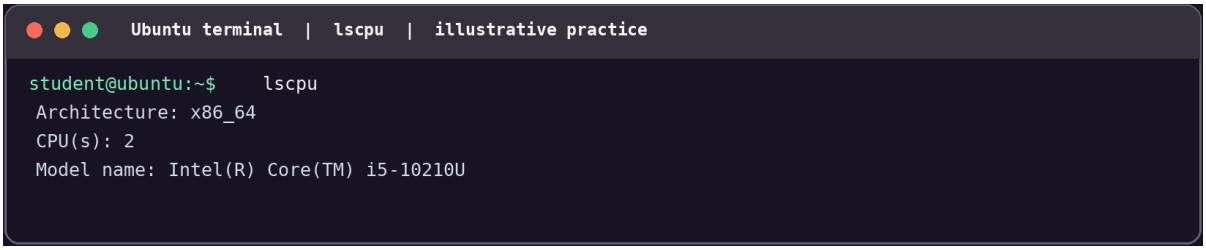
Syntax

`lscpu [options]`

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
-e	Show CPU information in a table	Function: show CPU information in a table. Example: lscpu -e
-J	Use JSON output on supported versions	Function: use JSON output on supported versions. Example: lscpu -J

Practice screenshot — test commands and example output



4.3.6 nproc

Definition

The nproc command prints the number of processing units available to the current process. It gives a quick numerical result that students can compare with the system load from uptime.

Why students use this command

Use it to compare a workload's CPU demand with available processing capacity.

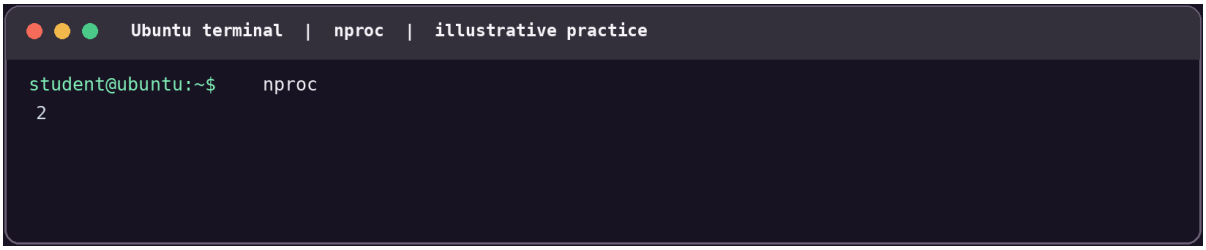
Syntax

```
nproc [options]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`--all`	Show all installed processing units when supported	Function: show all installed processing units when supported. Example: nproc --all

Practice screenshot — test commands and example output



4.3.7 lsblk

Definition

The lsblk (list block devices) command displays disks and other block devices in a tree. It can show disk size, partitions, and mount points. It is useful for identifying storage without changing disk contents.

Why students use this command

Use it to inspect storage layout without making changes.

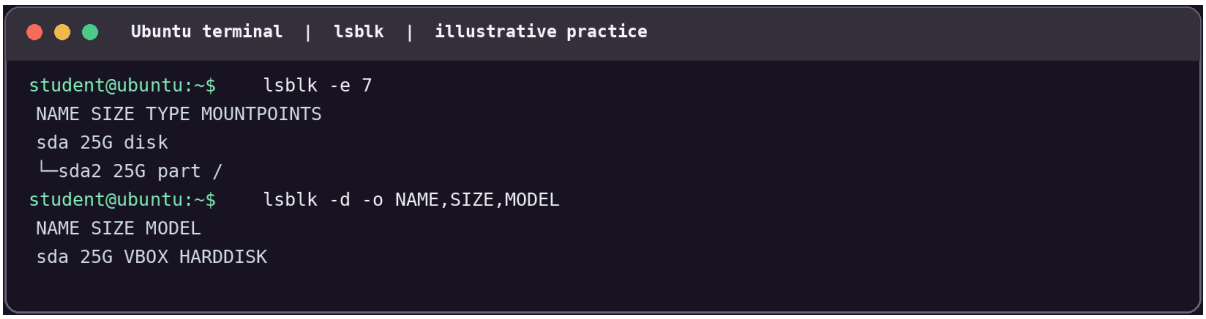
Syntax

```
lsblk [options]
```

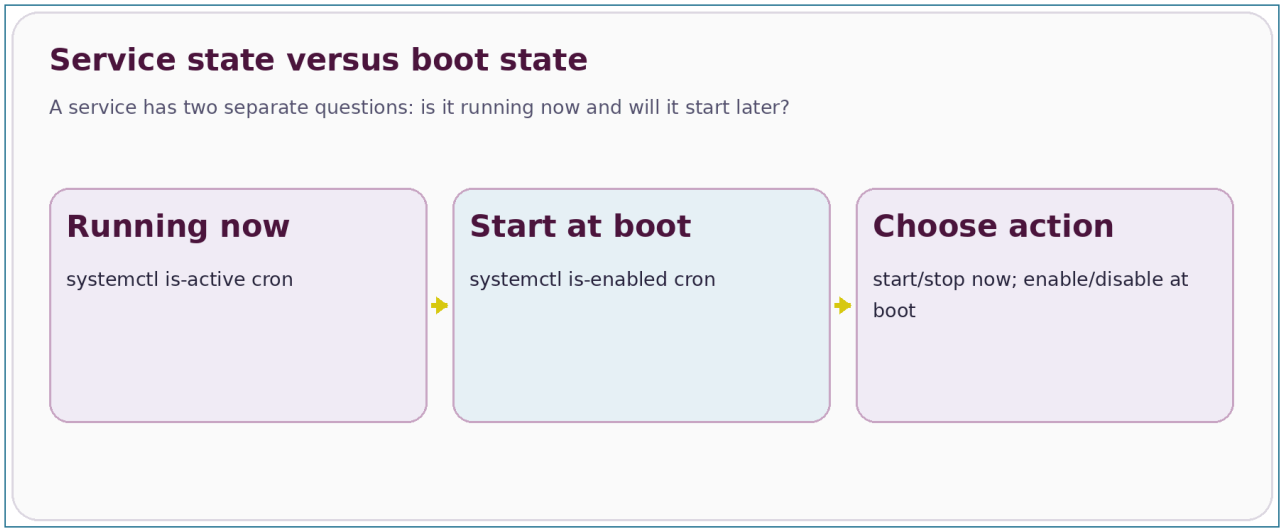
Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-e 7`	Exclude loop devices by major number	Function: exclude loop devices by major number. Example: lsblk -e 7
`-d`	Show whole disks only	Function: show whole disks only. Example: lsblk -d
`-o COLS`	Select output columns	Function: select output columns. Example: lsblk -o NAME,SIZE,TYPE,MOUNTPOINTS

Practice screenshot — test commands and example output



4.4 Services with systemd



systemd organises work in units. A .service unit describes how to run and manage a daemon; other unit types include .target for grouping dependencies and .timer for scheduled activation. The unit file contains sections such as [Unit], [Service], and [Install], each with configuration directives.

A service can be active but disabled, or inactive but enabled. For example, systemctl stop cron may make cron inactive for now while leaving its boot setting unchanged. Conversely, systemctl disable cron changes the boot configuration but usually does not stop an already running cron process.

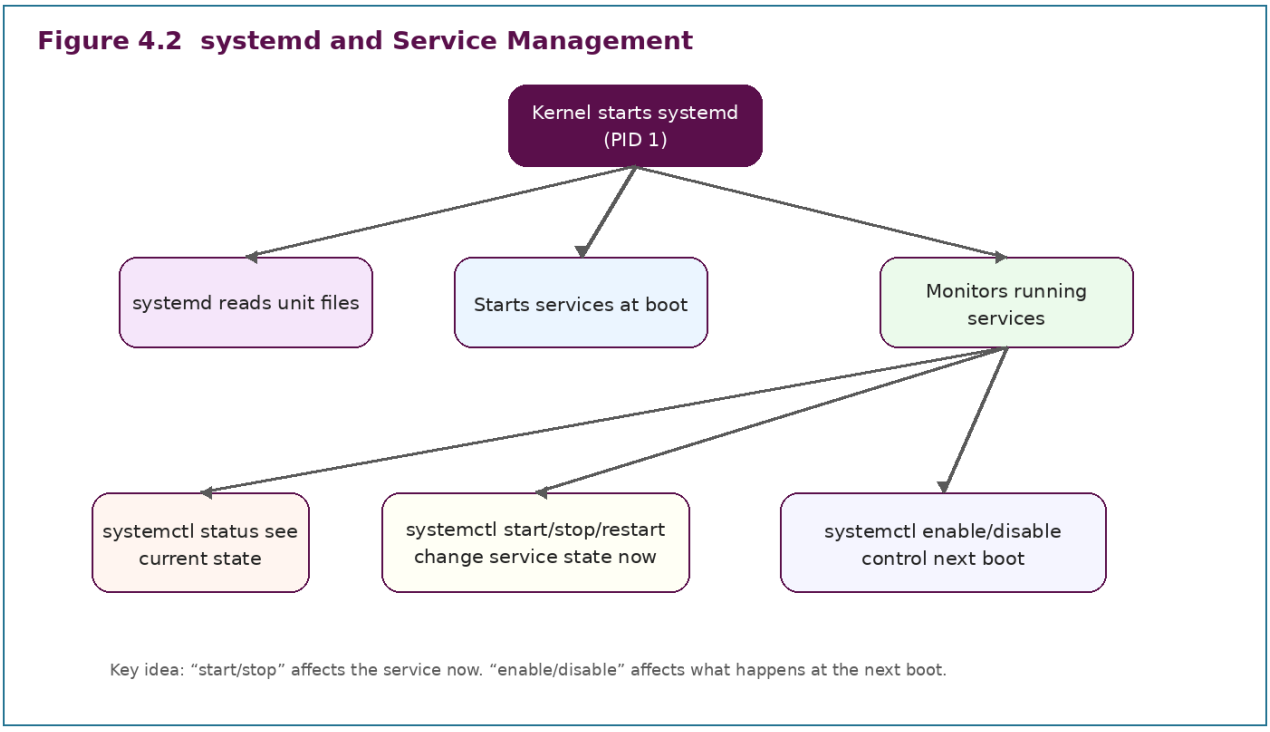


Figure 4.2 systemd as service manager and system startup manager.

4.4.1 systemctl

Definition

The systemctl command is the main tool for viewing and controlling systemd units, especially background services. It can report whether a service is currently running, start or stop it, and configure whether it starts during future boots. Remember that start/stop changes the service now, while enable/disable changes boot behaviour.

Why students use this command

Use it to check services before making any changes.

Syntax

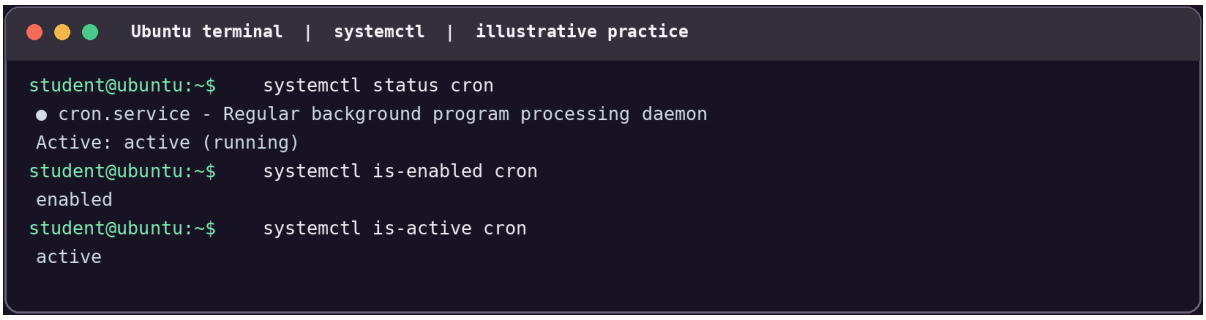
```
systemctl ACTION UNIT
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`status`	Read service health and recent messages	Function: read service health and recent messages. Example: systemctl status cron
`start`	Start a service now	Function: start a service now. Example: sudo systemctl start cron
`stop`	Stop a service now	Function: stop a service now. Example: sudo systemctl stop cron

Option / Shortcut	Name	Description & Example Usage
`restart`	Stop and start the service	Function: stop and start the service. Example: sudo systemctl restart cron
`reload`	Ask a supporting service to reread configuration	Function: ask a supporting service to reread configuration. Example: sudo systemctl reload ssh
`enable`	Configure service to start at boot	Function: configure service to start at boot. Example: sudo systemctl enable cron
`disable`	Stop automatic boot activation	Function: stop automatic boot activation. Example: sudo systemctl disable cron
`is-active`	Print current runtime state	Function: print current runtime state. Example: systemctl is-active cron
`is-enabled`	Print boot-activation state	Function: print boot-activation state. Example: systemctl is-enabled cron
`list-units`	List loaded units; may be filtered	Function: list loaded units; may be filtered. Example: systemctl list-units --type=service
`list-unit-files`	List installed unit definitions and enablement	Function: list installed unit definitions and enablement. Example: systemctl list-unit-files --type=service

Practice screenshot — test commands and example output



Important note. Start/stop applies now. Enable/disable applies to boot. Never stop SSH on a remote machine you reach only through SSH.

4.5 Logs, shutdown, and first troubleshooting

A useful troubleshooting sequence begins with observation rather than intervention. First inspect the status of the affected service or process, then read the logs from the relevant time range, and finally check configuration or resource conditions that could explain the failure.

Logs are time-stamped records, but they do not automatically prove the root cause of a problem. Different errors can be symptoms of the same underlying issue, such as low disk space, incorrect permissions, or an unavailable dependency. Compare timestamps and related messages before deciding on corrective action.

4.5.1 journalctl

Definition

The journalctl command reads the systemd journal, which stores messages from services, the kernel, and other components. It helps explain why something started, stopped, or failed. You can filter messages by service name, boot, time, or severity.

Why students use this command

Use it after systemctl status to investigate a service problem.

Syntax

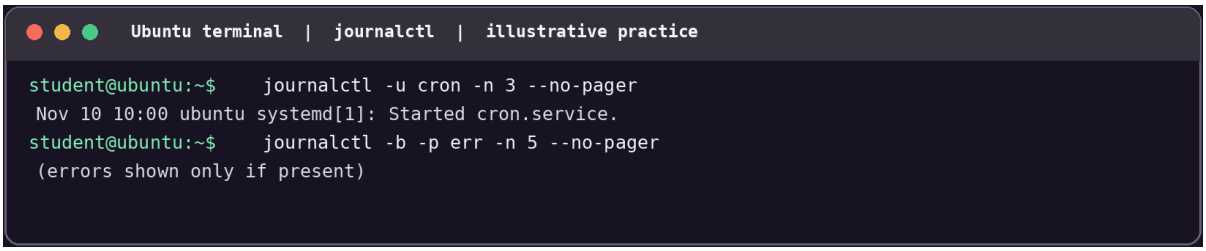
```
journalctl [options]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-u UNIT`	Show records for a given unit	Function: show records for a given unit. Example: journalctl -u cron
`-n N`	Show the most recent N entries	Function: show the most recent N entries. Example: journalctl -n 20

Option / Shortcut	Name	Description & Example Usage
`-b`	Restrict to the current boot	Function: restrict to the current boot. Example: journalctl -b
`-p err`	Show errors and higher-severity events	Function: show errors and higher-severity events. Example: journalctl -p err
`-f`	Follow new messages live	Function: follow new messages live. Example: journalctl -f
`--since TIME`	Restrict to records after a given time	Function: restrict to records after a given time. Example: journalctl --since "1 hour ago"

Practice screenshot — test commands and example output



4.5.2 dmesg

Definition

The dmesg command displays messages from the kernel ring buffer. These messages often describe hardware detection, drivers, disk events, and startup conditions. Ubuntu may restrict access, so sudo may be required.

Why students use this command

Use it when a device or boot problem may involve kernel messages.

Syntax

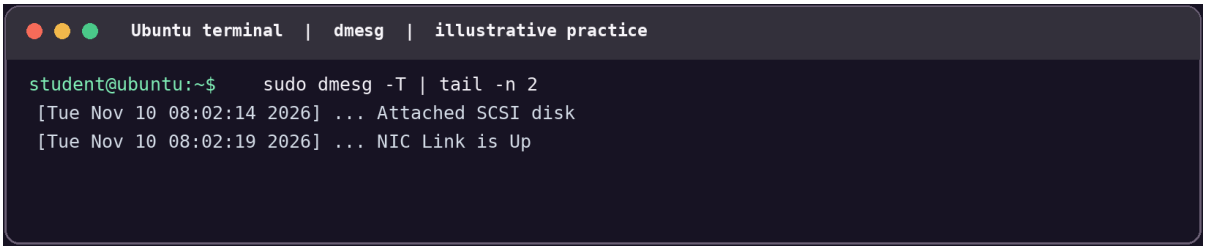
```
dmesg [options]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-T`	Display human-readable timestamps; approximate under some clock changes	Function: display human-readable timestamps; approximate under some clock changes. Example: sudo dmesg -T

Option / Shortcut	Name	Description & Example Usage
`--level=err`	Filter to error-level messages when supported	Function: filter to error-level messages when supported. Example: sudo dmesg --level=err

Practice screenshot — test commands and example output



4.5.3 shutdown

Definition

The shutdown command schedules a clean poweroff or reboot. It notifies the system and allows services to stop in an orderly way. You can choose an immediate action or schedule it for a later time, and cancel a pending shutdown with -c.

Why students use this command

Use it for planned system maintenance inside your own practice VM.

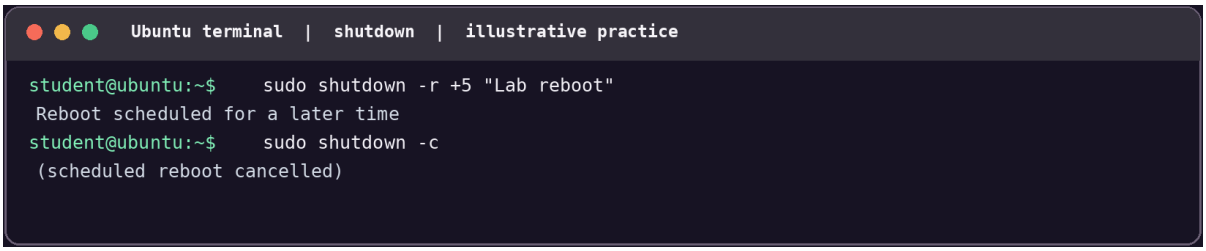
Syntax

```
sudo shutdown [options] TIME [message]
```

Options and shortcuts

Option / Shortcut	Name	Description & Example Usage
`-r`	Reboot rather than power off	Function: reboot rather than power off. Example: sudo shutdown -r now
`-c`	Cancel a pending scheduled shutdown	Function: cancel a pending scheduled shutdown. Example: sudo shutdown -c
`+N`	Schedule for N minutes from now	Function: schedule for N minutes from now. Example: sudo shutdown +5

Practice screenshot — test commands and example output



4.5.4 reboot

Definition

The reboot command asks the system to shut down services cleanly and start again. It is used after maintenance or changes that require a new boot. Do not run it on shared or remote machines without authorisation.

Why students use this command

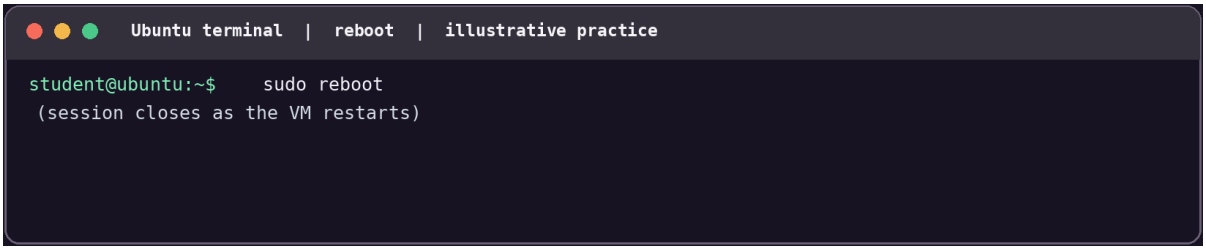
Use it only when you are ready for the VM to restart.

Syntax

```
sudo reboot
```

Options: The examples below use the command without additional flags.

Practice screenshot — test commands and example output



4.5.5 poweroff

Definition

The poweroff command requests a clean shutdown followed by powering down the machine. It should be preferred to abruptly cutting power to a virtual machine, because the operating system can save pending disk writes and stop services properly.

Why students use this command

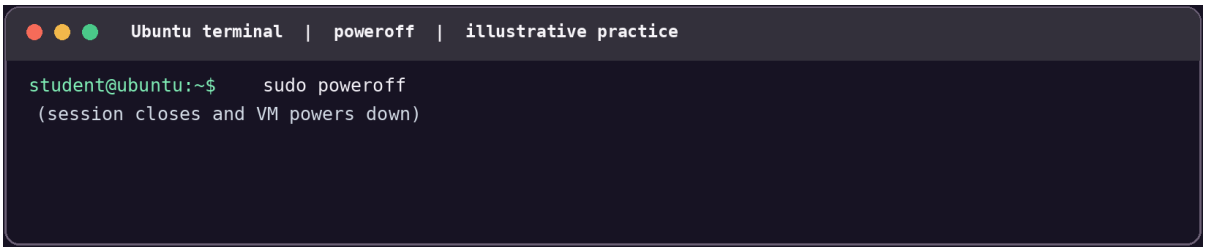
Use it to end a practice VM session cleanly.

Syntax

```
sudo poweroff
```

Options: The examples below use the command without additional flags.

Practice screenshot — test commands and example output



4.6 Guided troubleshooting patterns

When a service misbehaves, use a calm order:

6. Check the service status with systemctl status SERVICE.
7. Read more detailed logs with journalctl -u SERVICE -n 30 or journalctl -u SERVICE --since "1 hour ago".
8. Inspect the relevant configuration files under /etc and validate them if a validation command exists.
9. Reload or restart the service only after you understand the likely problem.

When the whole machine is slow, use another calm order:

10. Run uptime and compare the load average with the CPU count from nproc.
11. Run free -h to check available memory and swap usage.
12. Run top or htop to identify the processes using the most CPU or memory.
13. Decide whether to wait, renice, restart, or terminate the problem process.

Appendix A — Quick Command Reference

Task	Useful command examples
Find current directory	pwd
List files	ls, ls -lah
Change directory	cd /etc, cd ~, cd ..
Create folders/files	mkdir -p dir/subdir, touch file.txt
Copy or move	cp -r src dst, mv old new
Remove safely	rm -i file, rmdir emptydir
Read long text	less file, tail -f log
Search names or text	find . -name "*.txt", grep -in "word" file
Create archives	tar -czf backup.tar.gz folder/
Watch processes	ps -f, top, htop
Stop a process	kill PID, pkill name
Check system state	uname -a, uptime, free -h, lscpu, lsblk -e 7
Manage services	systemctl status/start/stop/restart SERVICE
Read logs	journalctl -u SERVICE -n 20, sudo dmesg -T

Appendix B — Glossary and Source-Coverage Checklist

Term	Clear meaning
Kernel	The central core of the operating system that manages hardware and system resources
Shell	A command interpreter such as Bash
PID	Process ID: the number that identifies a running process
PPID	Parent Process ID: the PID of the process that started another process
Environment variable	A named value used by the shell and programs, such as PATH or HOME
Service	A background program that waits to provide ongoing system work
systemd	The init and service-management system used by modern Ubuntu
Archive	A single file that groups many files together, often before compression
Symbolic link	A special file that points to another file by path

Source-coverage checklist: the revised book covers the four supplied chapter themes: Linux and Ubuntu foundations; Linux architecture, command line, and file management; text processing, searching, archiving, and editors; and process, service, and system management. Additional explanatory text, diagrams, and command examples were added to improve clarity and readability.

References and Further Reading

- Ubuntu Documentation — official Ubuntu guides and help pages.
- GNU Coreutils Manual — reference for common Linux command-line utilities.
- Linux man-pages project — manual pages for system calls and commands.
- systemd documentation — official information on systemd and systemctl.
- The Linux Foundation training resources — conceptual learning materials for Linux administration.